

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №4
на тему
**«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ.
КРИПТОСИСТЕМА МАК-ЭЛИСА»**

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Генерация ключей	7
2.2	Пошаговое зашифрование блока	8
2.3	Зашифрование и расшифрование текста	9
2.4	Работа с файлами	10
2.5	Обнаружение искажений	11
	Заключение	13
	Приложение А (обязательное) Листинг программного кода	14
	Приложение Б (обязательное) Блок-схема алгоритма	59

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения асимметричных криптосистем на примере криптосистемы Мак-Элиса, стойкость которой основана на сложности декодирования произвольного линейного кода. Задачи включают изучение математических основ криптосистемы – арифметики конечных полей, двоичных кодов Гоппы и алгоритма их декодирования, – проектирование и реализацию программного средства, осуществляющего генерацию ключевой пары, зашифрование открытым ключом и расшифрование закрытым ключом, а также тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая зашифрование и расшифрование данных криптосистемой Мак-Элиса как для текста, вводимого с клавиатуры, так и для файлов произвольного содержания.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Реализация разделена на модули, каждый из которых опирается только на нижележащие:

- `encryption::gf` – арифметика конечного поля $GF(2^m)$;
- `encryption::poly` – многочлены над $GF(2^m)$;
- `encryption::matrix` – векторы, матрицы и перестановки над $GF(2)$;
- `encryption::gorra` – построение двоичного кода Гоппы и декодер Паттерсона;
- `encryption::mceliece` – ключи, зашифрование и расшифрование блоков и сообщений.

Сторонние библиотеки для алгебры не используются, из внешних зависимостей подключена только библиотека `rand` для генерации случайных чисел. Взаимодействие с пользователем осуществляется через текстовое меню, общее с предыдущими лабораторными работами: оно позволяет выбрать источник данных (текст или файл), выполняемую операцию, просмотреть текущую ключевую пару и сгенерировать новую с выбранными параметрами.

Элементы поля $GF(2^m)$ представляются многочленами степени меньше m от переменной z с коэффициентами из $GF(2)$ и хранятся как m -битные числа. Сложение элементов – поразрядное исключающее ИЛИ, умножение – умножение многочленов по модулю неприводимого многочлена степени m . Для каждого $m = 2, \dots, 16$ в программе зафиксирован примитивный многочлен, например $z^{10} + z^3 + 1$ для $m = 10$, поэтому корень $\alpha = z$ порождает все ненулевые элементы поля. Это позволяет при создании поля построить таблицы степеней и логарифмов и выполнять умножение и обращение как сложение и вычитание показателей:

$$\alpha^i \cdot \alpha^j = \alpha^{(i+j) \bmod (2^m-1)}, \quad \sqrt{\alpha} = \alpha^{2^{m-1}}. \quad (2.1)$$

При построении таблиц проверяется, что степени α пробегает все ненулевые элементы, то есть что многочлен действительно примитивен.

Двоичный код Гоппы задаётся носителем $L = (\alpha_1, \dots, \alpha_n)$ из n различных элементов $GF(2^m)$ и неприводимым многочленом Гоппы $g(x)$ степени t с коэффициентами из $GF(2^m)$. Код состоит из двоичных векторов c длины n , для которых

$$S_c(x) = \sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)}. \quad (2.2)$$

Такой код имеет размерность $k \geq n - mt$ и минимальное расстояние не менее $2t + 1$, то есть исправляет до t ошибок. Проверочная матрица кода составляется из элементов $\alpha_i^j / g(\alpha_i)$, $j = 0, \dots, t - 1$; каждый элемент раскладывается на m бит, что даёт двоичную матрицу H размера $mt \times n$. Приведением H

методом Гаусса к виду $[A | I]$ получается порождающая матрица $G = [I_k | A^T]$ в систематическом виде, при котором первые k бит кодового слова совпадают с сообщением. Если при приведении приходится переставлять столбцы, так же переставляется и носитель L , иначе матрица G и декодер описывали бы разные коды.

Многочлен Гоппы выбирается случайно: генерируются случайные унитарные многочлены степени t , пока очередной не пройдет тест Бен-Ора, согласно которому многочлен g степени t над $GF(q)$ неприводим тогда и только тогда, когда $\gcd(g(x), x^{q^i} - x) = 1$ для всех $i = 1, \dots, \lfloor t/2 \rfloor$. Неприводимым оказывается примерно один многочлен из t , поэтому требуется в среднем около t попыток.

Ключевая пара строится из случайного кода Гоппы, случайной невырожденной матрицы S размера $k \times k$ и случайной перестановки P длины n :

$$G' = S \cdot G \cdot P. \quad (2.3)$$

Открытым ключом являются матрица G' и число t , закрытым – носитель L , многочлен $g(x)$, матрица S^{-1} и перестановка P^{-1} . Матрица S перемешивает базис кода, а P – координаты, поэтому G' внешне неотличима от порождающей матрицы случайного линейного кода. Невырожденная матрица находится перебором случайных матриц: обратимой оказывается около 29 % из них, а обратная вычисляется тем же методом Гаусса – Жордана. Параметры, предусмотренные в программе, приведены в таблице 2.1.

Таблица 2.1 – Наборы параметров криптосистемы

Набор	m	n	t	k	Размер G' , байт
toy	4	16	2	8	16
small	8	256	8	192	6144
original (1978)	10	1024	50	524	67 072

Зашифрование блока m длины k бит выполняется по формуле

$$c = m \cdot G' + e, \quad (2.4)$$

где e – случайный вектор длины n ровно с t единицами. Благодаря случайному вектору ошибок зашифрование вероятностное: одно и то же сообщение при каждом зашифровании даёт новый шифротекст.

Расшифрование начинается со снятия перестановки: $c \cdot P^{-1} = (mS) \cdot G + e \cdot P^{-1}$. Перестановка не меняет число единиц, поэтому полученный вектор u отличается от кодового слова mSG ровно в t позициях, и их находит алгоритм Паттерсона. Вычисляется синдром $S(x)$; если он нулевой, ошибок нет. Иначе ищется многочлен локаторов ошибок $\sigma(x) = \prod_{i \in E} (x - \alpha_i)$, корни которого – элементы носителя в позициях ошибок E . Он удовлетворяет ключевому

уравнению $S(x)\sigma(x) \equiv \sigma'(x) \pmod{g(x)}$. В характеристике 2 многочлен раскладывается как $\sigma = a^2 + xb^2$, откуда $\sigma' = b^2$, и уравнение сводится к

$$a(x) \equiv b(x)\tau(x) \pmod{g(x)}, \quad \tau(x) = \sqrt{S(x)^{-1} + x} \pmod{g(x)}. \quad (2.5)$$

Многочлены a и b находятся расширенным алгоритмом Евклида для g и τ , остановленным на первом остатке степени не выше $\lfloor t/2 \rfloor$; при этом степень b не превышает $\lfloor (t-1)/2 \rfloor$, и $\deg \sigma \leq t$. Квадратный корень по модулю неприводимого g вычисляется как $\sqrt{p} = p_0 + \sqrt{x} \cdot p_1$, где $p = p_0^2 + xp_1^2$, а $\sqrt{x} \pmod{g}$ находится один раз при создании ключа. Позиции ошибок определяются подстановкой всех элементов носителя в $\sigma(x)$, после чего соответствующие биты у инвертируются. Если число найденных корней не равно степени σ или исправленный вектор не является кодовым словом, ошибок больше t , и блок считается искажённым. Первые k бит исправленного слова равны mS , и сообщение восстанавливается умножением на S^{-1} . Блок-схема алгоритма приведена в приложении Б.

Сообщение переводится в последовательность бит (младший бит каждого байта первым), дополняется одной единицей и нулями до длины, кратной k , и разбивается на блоки по k бит. Дополнение добавляется всегда, поэтому при расшифровании оно снимается однозначно: отбрасываются последняя единица и нули за ней, причём единица должна стоять на границе байта в пределах последнего блока. Каждый шифроблок из n бит записывается в $\lceil n/8 \rceil$ байт; для параметров 1978 года это 128 байт на каждые 524 бита сообщения, то есть шифротекст примерно вдвое длиннее открытого текста.

Перед расшифрованием проверяется, что длина шифротекста ненулевая и кратна длине шифроблока, а биты за пределами n в последнем байте каждого блока нулевые. Если какой-либо блок не декодируется или дополнение оказывается некорректным, программа сообщает об искажении шифротекста.

Корректность реализации проверена 51 модульным тестом, покрывающим все уровни. Умножение по таблицам сверено с прямым умножением многочленов для всех пар элементов $\text{GF}(2^4)$ и $\text{GF}(2^8)$, тест неприводимости – с точной формулой числа неприводимых многочленов. Для кода $\text{GF}(2^4)$, $n = 16$, $t = 2$ перебором всех кодовых слов подтверждено минимальное расстояние не менее $2t + 1 = 5$. Декодер проверен на случайных векторах ошибок веса от 0 до t для нескольких наборов параметров, включая $n = 1024$, $t = 50$, а крипто-система в целом – зашифрованием и расшифрованием сообщений длиной от 0 до 257 байт, в том числе граничных длин, текстов на кириллице и файлов. Во всех случаях исходное сообщение восстанавливается без искажений.

Исходный код разработанного программного средства приведён в приложении А: арифметика поля представлена в листинге А.1, многочлены – в листинге А.2, матрицы – в листинге А.3, код Гоппы – в листинге А.4, крипто-система – в листинге А.5, точка входа в программу – в листинге А.6.

2.1 Генерация ключей

При запуске программа генерирует ключевую пару с параметрами из статьи Мак-Элиса 1978 года: $m = 10$, $n = 1024$, $t = 50$, $k = 524$. Генерация занимает около 15 мс. Меню программы и сведения о ключах представлены на рисунке 2.1: открытая матрица G' занимает 67 072 байта, матрица S^{-1} – 34 322 байта. Большие матрицы и списки программа выводит в сокращённом виде.

```
Generating keys for m = 10, n = 1024, t = 50...
Done in 12.15ms

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
k

Public key
n, k, t      : 1024, 524, 50
G' = S·G·P   : 524 × 1024, 67072 bytes
Private key
field        : GF(2^10) = GF(2)[z]/(z^10 + z^3 + 1)
g(x)         : x^50 + α^402·x^49 + α^727·x^48 + α^18·x^47 + α^529·x^46 + α^1012·x^45 +
α^658·x^44 + α^464·x^43 + α^568·x^42 + α^710·x^41 + α^910·x^40 + α^949·x^39 + α^339·x^38 +
α^418·x^37 + α^735·x^36 + α^103·x^35 + α^899·x^34 + α^374·x^33 + α^254·x^32 + α^696·x^31 +
α^334·x^30 + α^539·x^29 + α^734·x^28 + α^912·x^27 + α^877·x^26 + α^638·x^25 + α^380·x^24 +
α^190·x^23 + α^885·x^22 + α^411·x^21 + α^850·x^20 + α^939·x^19 + α^818·x^18 + α^61·x^17 +
α^464·x^16 + α^676·x^15 + α^246·x^14 + α^601·x^13 + α^539·x^12 + α^107·x^11 + α^530·x^10 +
α^1014·x^9 + α^832·x^8 + α^1001·x^7 + α^840·x^6 + α^209·x^5 + α^197·x^4 + α^251·x^3 +
α^731·x^2 + α^941·x + α^390
support L     : α^977, α^102, α^883, α^198, α^827, α^880, α^340, α^829, α^569, α^719, α^240,
α^546, α^892, α^297, α^425, α^901, α^104, α^836, α^249, α^414, α^725, α^736, α^428, α^103,
α^720, α^791, α^850, α^404, α^413, α^779, α^309, α^170, α^782, α^355, α^421, α^682, α^818,
α^463, α^982, α^391, α^377, α^574, α^911, α^873, α^959, α^472, α^369, α^516, α^341, α^279,
α^746, α^56, α^512, α^974, α^831, α^346, α^914, α^878, α^283, α^603, α^895, α^490, α^378,
α^255, ... (960 more)
S^-1         : 524 × 524, 34322 bytes
P^-1         : 969, 86, 987, 286, 144, 797, 905, 620, 725, 168, 187, 808, 621, 266, 457, 847,
579, 927, 21, 979, 339, 855, 43, 492, 774, 1004, 888, 644, 839, 607, 878, 971, 145, 822,
1021, 352, 915, 319, 514, 720, 106, 98, 530, 633, 91, 447, 395, 54, 597, 887, 683, 181,
112,
646, 513, 781, 265, 140, 823, 864, 336, 75, 1, 388, ... (960 more)
```

Рисунок 2.1 – Меню программы и ключевая пара с параметрами 1978 года

Команда генерации ключей позволяет выбрать один из наборов параметров таблицы 2.1 или задать m , n , t вручную. Введённые параметры проверяются: m должно лежать в диапазоне от 2 до 16, длина кода не может превышать число элементов поля 2^m , а mt должно быть меньше n , чтобы в блоке оставалось место для данных. При ошибке сохраняется прежняя ключевая пара. На рисунке 2.2 показаны отказ для $n = 20 > 2^4$ и ключи учебного набора, у которого все матрицы выводятся полностью. Многочлен Гоппы и элементы носителя записаны как степени примитивного элемента α .

```

McEliece cryptosystem - input/output: file, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
g

1) toy              m = 4 , n = 16 , t = 2
2) small            m = 8 , n = 256 , t = 8
3) original (1978) m = 10, n = 1024, t = 50
4) custom
Parameters [keep current]: 4
m (2..=16) [10]: 4
n (up to 2^m = 16) [1024]: 20
t [50]: 2
Generating keys for m = 4, n = 20, t = 2...
Error: n = 20 exceeds the field size 2^4

McEliece cryptosystem - input/output: file, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
g

1) toy              m = 4 , n = 16 , t = 2
2) small            m = 8 , n = 256 , t = 8
3) original (1978) m = 10, n = 1024, t = 50
4) custom
Parameters [keep current]: 1
Generating keys for m = 4, n = 16, t = 2...
Done in 24.10µs

McEliece cryptosystem - input/output: file, n = 16, k = 8, t = 2
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
k

Public key
n, k, t : 16, 8, 2
G' = S·G·P : 8 × 16, 16 bytes
          10111011 10011001
          00110010 01011101
          10000110 00101001
          10111000 11010011
          10101110 10100010
          11011010 10000000
          00101101 11010111
          11110000 11101111

Private key
field : GF(2^4) = GF(2)[z]/(z^4 + z + 1)
g(x) : x^2 + α^9·x + α
support L : α^12, α^2, α^11, 1, α^7, α^10, α^14, 0, α^9, α^5, α^8, α^6, α^3, α^13, α, α^4
S^-1 : 8 × 8, 8 bytes
      11010010
      00111011
      01111101
      01000101
      00101001
      10001011
      11010110
      00001100
P^-1 : 14, 10, 7, 12, 15, 9, 6, 1, 0, 11, 8, 3, 2, 4, 13, 5

```

Рисунок 2.2 – Проверка параметров и ключи учебного набора

2.2 Пошаговое зашифрование блока

Для наглядности программа позволяет провести один блок через все этапы алгоритма с выводом промежуточных значений (рисунок 2.3). Случайный вектор ошибок имеет единицы в позициях 8 и 9. После снятия перестановки

P они переходят в позиции 0 и 11, которые и находит декодер Паттерсона по синдрому $S(x) = \alpha^{10}x + \alpha^8$. Первые 8 бит исправленного кодового слова равны mS , умножение на S^{-1} возвращает исходное сообщение.

```
McEliece cryptosystem - input/output: file, n = 16, k = 8, t = 2
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
3

Message (8 bits, empty for random):

Encryption
m           : 10111000
m·G'        : 00101011 11000001
e (weight 2) : 00000000 11000000
e positions  : [8, 9]
c = m·G' + e : 00101011 00000001

Decryption
c·P-1       : 01000111 00000001
syndrome S(x) : α10·x + α8
fixed at      : [0, 11] (e·P-1)
m·S·G         : 11000111 00010001
m·S (first k) : 11000111
m = m·S·S-1 : 10111000
result        : matches the message
block sizes   : 8 bits in, 16 bits out
```

Рисунок 2.3 – Пошаговое зашифрование и расшифрование одного блока

2.3 Зашифрование и расшифрование текста

Результат работы программы для текста, вводимого с клавиатуры, представлен на рисунке 2.4. Сообщение длиной 30 байт вместе с битом дополнения укладывается в один блок из 524 бит, поэтому шифротекст занимает 128 байт. Повторное зашифрование того же сообщения тем же ключом даёт другой шифротекст, так как каждый раз выбирается новый вектор ошибок; оба шифротекста расшифровываются в исходное сообщение.

```

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
1

Plaintext: Привет, Мак-Элис!
mode      : encryption
plaintext : 30 bytes
encrypted :
29548F7A750D8A3750F3DB55B59337078C78834EBA0599E7D6200476AB58B382B1682229467370D041

1F6EA7D970C50E7903C7616164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF83B
0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682D0 (128
bytes)

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
1

Plaintext: Привет, Мак-Элис!
mode      : encryption
plaintext : 30 bytes
encrypted :
295E871AFD0D823710F3DE57F49337078C58834EBA0599E3F2300674ABD82682A068A3205473704081

1F6CA2D950C50E7903DF616164F1C1B54D4D6FF0E20D69ED9962571C037CC8CD33DA7B0B83F8184383B5A97E883B
0E010D2E9FBAE9D82847C576DD1E3F012F329E06FFF90B3036E39AF2A14F9F806EE202352254F7C2D0 (128
bytes)

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
2

Ciphertext (hex):
29548F7A750D8A3750F3DB55B59337078C78834EBA0599E7D6200476AB58B382B1682229467370

D0411F6EA7D970C50E7903C7616164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF8
3B0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682D0
mode      : decryption
encrypted : 128 bytes
decrypted : Привет, Мак-Элис! (30 bytes)

```

Рисунок 2.4 – Зашифрование и расшифрование текста

2.4 Работа с файлами

Результат зашифрования и расшифрования файла представлен на рисунке 2.5. Файл длиной 1800 байт, или 14 400 бит, после дополнения разбивается на 28 блоков, и шифротекст занимает $28 \cdot 128 = 3584$ байта. По умолчанию к имени зашифрованного файла добавляется расширение .mse, а при расшифровании оно отбрасывается; если файл с таким именем уже существует, программа запрашивает подтверждение перезаписи. Расшифрованный файл побайтно совпадает с исходным.

```

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
m

Switched to file mode

McEliece cryptosystem - input/output: file, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
1

Plaintext file: plain.txt
Output file [plain.txt.mce]:
mode      : encryption
input     : plain.txt (1800 bytes)
output    : plain.txt.mce (3584 bytes)

McEliece cryptosystem - input/output: file, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
2

Ciphertext file: plain.txt.mce
Output file [plain.txt]: out.txt
mode      : decryption
input     : plain.txt.mce (3584 bytes)
output    : out.txt (1800 bytes)

```

Рисунок 2.5 – Зашифрование и расшифрование файла

2.5 Обнаружение искажений

Реакция программы на искажённый шифротекст показана на рисунке 2.6. В первом шифротексте изменён один бит: к 50 ошибкам, внесённым при зашифровании, добавилась 51-я, и декодер, исправляющий не более $t = 50$ ошибок, отвергает блок. Если бы изменённый бит совпал с одной из позиций вектора ошибок, их осталось бы 49 и блок расшифровался бы верно. Второй шифротекст укорочен на один байт и отвергается ещё до расшифрования, поскольку его длина не кратна длине шифроблока.

```

McEliece cryptosystem – input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
2

Ciphertext (hex):
29548F7A750D8A3750F3DB55B59337078C78834EBB0599E7D6200476AB58B382B1682229467370

D0411F6EA7D970C50E7903C7616164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF8
3B0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682D0
Error: Block corrupted

McEliece cryptosystem – input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit):
2

Ciphertext (hex):
29548F7A750D8A3750F3DB55B59337078C78834EBA0599E7D6200476AB58B382B1682229467370

D0411F6EA7D970C50E7903C7616164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF8
3B0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682
Error: Invalid message length

```

Рисунок 2.6 – Расшифрование искажённого шифротекста

Таким образом криптосистема Мак-Элиса была изучена, реализована и протестирована в полном объёме как для текста, вводимого с клавиатуры, так и для файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения асимметричных криптосистем на примере криптосистемы Мак-Элиса. Было разработано программное средство на языке *Rust*, реализующее арифметику конечных полей $GF(2^m)$, многочленов над ними и матриц над $GF(2)$, построение двоичных кодов Гоппы и их декодирование алгоритмом Паттерсона, а на их основе – генерацию ключевой пары, зашифрование открытым ключом и расшифрование закрытым ключом над текстом, вводимым с клавиатуры, либо над файлами произвольного содержания.

Тестирование показало, что расшифрование восстанавливает исходное сообщение без искажений при любой длине, а искажение шифротекста, вносящее больше t ошибок в блок, обнаруживается. Работа показала основные особенности криптосистемы. Её стойкость основана не на теории чисел, а на NP-трудной задаче декодирования произвольного линейного кода, для которой неизвестны эффективные алгоритмы, в том числе квантовые, поэтому криптосистема относится к постквантовым. Зашифрование сводится к умножению вектора на матрицу и сложению с вектором ошибок и выполняется очень быстро, а зашифрование вероятностное благодаря случайному вектору ошибок.

Отдельно следует отметить отличия от изученной ранее криптосистемы Рабина: преобразования выполняются не над большими целыми числами, а над битовыми векторами и многочленами над конечными полями; открытый ключ очень велик и для параметров 1978 года занимает 67 072 байта против 384 байт модуля Рабина; шифротекст примерно вдвое длиннее открытого текста; наконец, для однозначного расшифрования не требуется избыточность, так как декодер восстанавливает единственное кодовое слово.

Все задачи лабораторной работы выполнены в полном объёме.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Арифметика поля $GF(2^m)$

```
use rand::Rng;

/// An element of  $GF(2^m)$ : bit i is the coefficient of  $z^i$  in a polynomial of
degree < m.
pub type Gf = u16;

pub const MIN_M: u32 = 2;
pub const MAX_M: u32 = 16;

/// Primitive polynomials for  $m = 2..=16$ ; bit i is the coefficient of  $z^i$ .
const PRIMITIVE_POLYNOMIALS: [u32; 15] = [
    0x7,      //  $z^2 + z + 1$ 
    0xB,      //  $z^3 + z + 1$ 
    0x13,     //  $z^4 + z + 1$ 
    0x25,     //  $z^5 + z^2 + 1$ 
    0x43,     //  $z^6 + z + 1$ 
    0x83,     //  $z^7 + z + 1$ 
    0x11D,    //  $z^8 + z^4 + z^3 + z^2 + 1$ 
    0x211,    //  $z^9 + z^4 + 1$ 
    0x409,    //  $z^{10} + z^3 + 1$ 
    0x805,    //  $z^{11} + z^2 + 1$ 
    0x1053,   //  $z^{12} + z^6 + z^4 + z + 1$ 
    0x201B,   //  $z^{13} + z^4 + z^3 + z + 1$ 
    0x4443,   //  $z^{14} + z^{10} + z^6 + z + 1$ 
    0x8003,   //  $z^{15} + z + 1$ 
    0x1100B,  //  $z^{16} + z^{12} + z^3 + z + 1$ 
];

///  $GF(2^m)$  built from a primitive polynomial, with multiplication through
log/exp tables.
pub struct Field {
    m: u32,
    modulus: u32,
    ///  $\exp[i] = \alpha^i$ , stored twice over so that a sum of two logarithms needs
no reduction.
    exp: Vec<Gf>,
    ///  $\log[a] = i$  such that  $\alpha^i = a$ ;  $\log[0]$  is meaningless.
    log: Vec<u32>,
}

impl Field {
    pub fn new(m: u32) -> Self {
        assert!(
            (MIN_M..=MAX_M).contains(&m),
            "m must be in {MIN_M}..={MAX_M}"
        );

        return Self::with_modulus(m, PRIMITIVE_POLYNOMIALS[(m - MIN_M) as
usize])
            .expect("the built-in polynomials are primitive");
    }

    pub fn with_modulus(m: u32, modulus: u32) -> Result<Self, String> {
        if !(MIN_M..=MAX_M).contains(&m) {
            return Err(format!("m must be in {MIN_M}..={MAX_M}"));
        }
    }
}
```

```

        if modulus >> m != 1 {
            return Err(format!("{modulus:#b} is not a polynomial of degree
{m}"));
        }

        let order = (1usize << m) - 1;
        let mut exp = vec![0; 2 * order];
        let mut log = vec![0; order + 1];
        let mut seen = vec![false; order + 1];

        //  $\alpha$  is primitive exactly when its powers run through every non-zero
        element.
        let mut a: u32 = 1;
        for i in 0..order {
            if a == 0 || seen[a as usize] {
                return Err(format!("{modulus:#b} is not primitive"));
            }
            seen[a as usize] = true;
            exp[i] = a as Gf;
            exp[i + order] = a as Gf;
            log[a as usize] = i as u32;

            a <=< 1;
            if a >> m != 0 {
                a ^= modulus;
            }
        }
        if a != 1 {
            return Err(format!("{modulus:#b} is not primitive"));
        }

        return Ok(Self {
            m,
            modulus,
            exp,
            log,
        });
    }

    pub fn m(&self) -> u32 {
        return self.m;
    }

    pub fn modulus(&self) -> u32 {
        return self.modulus;
    }

    /// Number of elements,  $2^m$ .
    pub fn size(&self) -> usize {
        return 1 << self.m;
    }

    /// Order of the multiplicative group,  $2^m - 1$ .
    pub fn order(&self) -> usize {
        return self.size() - 1;
    }

    pub fn add(&self, a: Gf, b: Gf) -> Gf {
        return a ^ b;
    }

    pub fn mul(&self, a: Gf, b: Gf) -> Gf {
        if a == 0 || b == 0 {

```

```

        return 0;
    }

    return self.exp[(self.log[a as usize] + self.log[b as usize]) as
usize];
}

pub fn inv(&self, a: Gf) -> Gf {
    assert!(a != 0, "zero has no inverse");

    return self.exp[(self.order() - self.log[a as usize] as usize) %
self.order()];
}

pub fn div(&self, a: Gf, b: Gf) -> Gf {
    return self.mul(a, self.inv(b));
}

pub fn square(&self, a: Gf) -> Gf {
    return self.mul(a, a);
}

/// The square root, which always exists and is unique in characteristic
2.
pub fn sqrt(&self, a: Gf) -> Gf {
    if a == 0 {
        return 0;
    }

    //  $\sqrt{a} = a^{(2^{m-1})}$ , because squaring it gives  $a^{(2^m)} = a$ .
    let exponent = self.log[a as usize] as u64 * (1 << (self.m - 1)) %
self.order() as u64;

    return self.exp[exponent as usize];
}

pub fn pow(&self, a: Gf, e: u64) -> Gf {
    if a == 0 {
        return if e == 0 { 1 } else { 0 };
    }

    let order = self.order() as u64;
    let exponent = self.log[a as usize] as u64 * (e % order) % order;

    return self.exp[exponent as usize];
}

///  $\alpha^i$  for the primitive element  $\alpha = z$ .
pub fn alpha_pow(&self, i: usize) -> Gf {
    return self.exp[i % self.order()];
}

pub fn log(&self, a: Gf) -> Option<u32> {
    if a == 0 {
        return None;
    }

    return Some(self.log[a as usize]);
}

pub fn random<R: Rng>(&self, rng: &mut R) -> Gf {
    return rng.gen_range(0..self.size()) as Gf;
}

```



```

pub fn random_nonzero<R: Rng>(&self, rng: &mut R) -> Gf {
    return rng.gen_range(1..self.size()) as Gf;
}

/// The element as a power of  $\alpha$ : "0", "1", " $\alpha$ " or " $\alpha^i$ ".
pub fn display(&self, a: Gf) -> String {
    return match self.log(a) {
        None => "0".to_string(),
        Some(0) => "1".to_string(),
        Some(1) => " $\alpha$ ".to_string(),
        Some(i) => format!(" $\alpha^{{i}}$ "),
    };
}

}

#[cfg(test)]
mod tests {
    use super::*;

    /// Shift-and-add multiplication, independent of the tables.
    fn mul_reference(a: Gf, b: Gf, m: u32, modulus: u32) -> Gf {
        let mut a = a as u32;
        let mut b = b as u32;
        let mut result = 0;
        while b != 0 {
            if b & 1 == 1 {
                result ^= a;
            }
            b >>= 1;
            a <<= 1;
            if a >> m != 0 {
                a ^= modulus;
            }
        }

        return result as Gf;
    }

    #[test]
    fn built_in_polynomials_are_primitive() {
        for m in MIN_M..=MAX_M {
            let field = Field::new(m);
            assert_eq!(field.size(), 1 << m);
            assert_eq!(field.alpha_pow(field.order()), 1);
        }
    }

    #[test]
    fn rejects_bad_moduli() {
        //  $z^4 + z^3 + z^2 + z + 1$ : irreducible, but  $\alpha$  has order 5.
        assert!(Field::with_modulus(4, 0b11111).is_err());
        //  $z^4 + z^2 + 1 = (z^2 + z + 1)^2$ .
        assert!(Field::with_modulus(4, 0b10101).is_err());
        //  $z^4 + z^3 = z^3(z + 1)$ .
        assert!(Field::with_modulus(4, 0b11000).is_err());
        // Wrong degree.
        assert!(Field::with_modulus(4, 0b1011).is_err());
        assert!(Field::with_modulus(4, 0b10011).is_ok());
    }

    #[test]
    fn mul_matches_reference() {

```

```

    for m in [4, 8] {
        let field = Field::new(m);
        for a in 0..field.size() as Gf {
            for b in 0..field.size() as Gf {
                assert_eq!(field.mul(a, b), mul_reference(a, b, m,
field.modulus()));
            }
        }
    }

}

#[test]
fn distributive() {
    let field = Field::new(4);
    for a in 0..16 {
        for b in 0..16 {
            for c in 0..16 {
                assert_eq!(
                    field.mul(a, field.add(b, c)),
                    field.add(field.mul(a, b), field.mul(a, c))
                );
            }
        }
    }
}

#[test]
fn inverse_and_division() {
    let field = Field::new(8);
    for a in 1..field.size() as Gf {
        assert_eq!(field.mul(a, field.inv(a)), 1);
        for b in [1, 2, 0x53, 0xFF] {
            assert_eq!(field.mul(field.div(a, b), b), a);
        }
    }
}

#[test]
fn sqrt_inverts_square() {
    for m in [4, 8, 13] {
        let field = Field::new(m);
        for a in 0..field.size() as Gf {
            assert_eq!(field.square(field.sqrt(a)), a);
            assert_eq!(field.sqrt(field.square(a)), a);
        }
    }
}

#[test]
fn pow() {
    let field = Field::new(8);
    assert_eq!(field.pow(0, 0), 1);
    assert_eq!(field.pow(0, 5), 0);
    for a in 1..field.size() as Gf {
        assert_eq!(field.pow(a, 0), 1);
        assert_eq!(field.pow(a, 1), a);
        assert_eq!(field.pow(a, 2), field.square(a));
        assert_eq!(field.pow(a, 3), field.mul(a, field.square(a)));
        assert_eq!(field.pow(a, field.order() as u64), 1);
    }
}

#[test]

```

```

fn display() {
    let field = Field::new(4);
    assert_eq!(field.display(0), "0");
    assert_eq!(field.display(1), "1");
    assert_eq!(field.display(0b10), "α");
    // α^7 = z^3 + z + 1 for z^4 + z + 1.
    assert_eq!(field.display(0b1011), "α^7");
}
}

```

Листинг А.2 – Многочлены над $GF(2^m)$

```

use super::gf::{Field, Gf};
use rand::Rng;

/// A polynomial in x over  $GF(2^m)$ . coeffs[i] is the coefficient of  $x^i$  and
the last
/// coefficient is never zero, so the zero polynomial has no coefficients at
all.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Poly {
    coeffs: Vec<Gf>,
}

impl Poly {
    pub fn zero() -> Self {
        return Self { coeffs: vec![] };
    }

    pub fn one() -> Self {
        return Self::constant(1);
    }

    pub fn x() -> Self {
        return Self::monomial(1, 1);
    }

    pub fn constant(c: Gf) -> Self {
        return Self::from_coeffs(vec![c]);
    }

    ///  $c \cdot x^{\text{degree}}$ .
    pub fn monomial(c: Gf, degree: usize) -> Self {
        let mut coeffs = vec![0; degree + 1];
        coeffs[degree] = c;

        return Self::from_coeffs(coeffs);
    }

    pub fn from_coeffs(coeffs: Vec<Gf>) -> Self {
        let mut poly = Self { coeffs };
        poly.normalize();

        return poly;
    }

    fn normalize(&mut self) {
        while self.coeffs.last() == Some(&0) {
            self.coeffs.pop();
        }
    }

    pub fn coeffs(&self) -> &[Gf] {

```

```

        return &self.coeffs;
    }

    pub fn coeff(&self, i: usize) -> Gf {
        return self.coeffs.get(i).copied().unwrap_or(0);
    }

    /// None for the zero polynomial.
    pub fn degree(&self) -> Option<usize> {
        return self.coeffs.len().checked_sub(1);
    }

    pub fn is_zero(&self) -> bool {
        return self.coeffs.is_empty();
    }

    pub fn leading(&self) -> Gf {
        return self.coeffs.last().copied().unwrap_or(0);
    }

    pub fn is_monic(&self) -> bool {
        return self.leading() == 1;
    }

    /// Addition and subtraction coincide in characteristic 2.
    pub fn add(&self, other: &Poly) -> Poly {
        let (long, short) = if self.coeffs.len() >= other.coeffs.len() {
            (self, other)
        } else {
            (other, self)
        };
        let mut coeffs = long.coeffs.clone();
        for (c, &s) in coeffs.iter_mut().zip(&short.coeffs) {
            *c ^= s;
        }

        return Self::from_coeffs(coeffs);
    }

    pub fn scale(&self, c: Gf, field: &Field) -> Poly {
        return Self::from_coeffs(self.coeffs.iter().map(|&a| field.mul(a, c)).collect());
    }

    pub fn mul(&self, other: &Poly, field: &Field) -> Poly {
        if self.is_zero() || other.is_zero() {
            return Self::zero();
        }

        let mut coeffs = vec![0; self.coeffs.len() + other.coeffs.len() - 1];
        for (i, &a) in self.coeffs.iter().enumerate() {
            if a == 0 {
                continue;
            }
            for (j, &b) in other.coeffs.iter().enumerate() {
                coeffs[i + j] ^= field.mul(a, b);
            }
        }

        return Self::from_coeffs(coeffs);
    }

    /// In characteristic 2 the cross terms cancel:  $(\sum c_i x^i)^2 = \sum c_i^2 x^{2i}$ .

```

```

pub fn square(&self, field: &Field) -> Poly {
    let mut coeffs = vec![0; (2 * self.coeffs.len()).saturating_sub(1)];
    for (i, &c) in self.coeffs.iter().enumerate() {
        coeffs[2 * i] = field.square(c);
    }

    return Self::from_coeffs(coeffs);
}

/// Quotient and remainder of long division; panics on a zero divisor.
pub fn div_rem(&self, divisor: &Poly, field: &Field) -> (Poly, Poly) {
    let divisor_degree = divisor.degree().expect("division by the zero polynomial");
    if self.coeffs.len() <= divisor_degree {
        return (Self::zero(), self.clone());
    }

    let leading_inv = field.inv(divisor.leading());
    let mut rem = self.coeffs.clone();
    let mut quot = vec![0; rem.len() - divisor_degree];
    for i in (0..quot.len()).rev() {
        let c = field.mul(rem[i + divisor_degree], leading_inv);
        if c == 0 {
            continue;
        }
        quot[i] = c;
        for (j, &d) in divisor.coeffs.iter().enumerate() {
            rem[i + j] ^= field.mul(c, d);
        }
    }
    rem.truncate(divisor_degree);

    return (Self::from_coeffs(quot), Self::from_coeffs(rem));
}

pub fn rem(&self, modulus: &Poly, field: &Field) -> Poly {
    return self.div_rem(modulus, field).1;
}

/// Horner's scheme.
pub fn eval(&self, x: Gf, field: &Field) -> Gf {
    return self
        .coeffs
        .iter()
        .rev()
        .fold(0, |acc, &c| field.mul(acc, x) ^ c);
}

pub fn monic(&self, field: &Field) -> Poly {
    if self.is_zero() {
        return Self::zero();
    }

    return self.scale(field.inv(self.leading()), field);
}

/// Monic greatest common divisor; gcd(0, 0) = 0.
pub fn gcd(&self, other: &Poly, field: &Field) -> Poly {
    let mut a = self.clone();
    let mut b = other.clone();
    while !b.is_zero() {
        let r = a.rem(&b, field);
        a = b;
    }
}

```

```

        b = r;
    }

    return a.monic(field);
}

pub fn mul_mod(&self, other: &Poly, modulus: &Poly, field: &Field) ->
Poly {
    return self.mul(other, field).rem(modulus, field);
}

pub fn square_mod(&self, modulus: &Poly, field: &Field) -> Poly {
    return self.square(field).rem(modulus, field);
}

/// The inverse modulo `modulus`, if the two are coprime.
pub fn inv_mod(&self, modulus: &Poly, field: &Field) -> Option<Poly> {
    // Extended Euclid keeping  $r_i \equiv v_i \cdot \text{self} \pmod{\text{modulus}}$ .
    let (mut r0, mut r1) = (modulus.clone(), self.rem(modulus, field));
    let (mut v0, mut v1) = (Self::zero(), Self::one());
    while !r1.is_zero() {
        let (q, r) = r0.div_rem(&r1, field);
        let v = v0.add(&q.mul(&v1, field));
        (r0, r1) = (r1, r);
        (v0, v1) = (v1, v);
    }
    if r0.degree() != Some(0) {
        return None;
    }

    return Some(v0.scale(field.inv(r0.leading()), field).rem(modulus,
field));
}

/// Runs extended Euclid on (modulus, self) up to the first remainder r
with
    ///  $\deg r \leq \text{stop\_degree}$  and returns (r, v) such that  $r \equiv v \cdot \text{self} \pmod{\text{modulus}}$ .
    /// Each v then has  $\deg v = \deg \text{modulus} - \deg(\text{previous remainder}) < \deg \text{modulus} - \text{stop\_degree}$ .
    pub fn partial_ext_gcd(
        &self,
        modulus: &Poly,
        stop_degree: usize,
        field: &Field,
    ) -> (Poly, Poly) {
        let (mut r0, mut r1) = (modulus.clone(), self.rem(modulus, field));
        let (mut v0, mut v1) = (Self::zero(), Self::one());
        while r1.degree().is_some_and(|d| d > stop_degree) {
            let (q, r) = r0.div_rem(&r1, field);
            let v = v0.add(&q.mul(&v1, field));
            (r0, r1) = (r1, r);
            (v0, v1) = (v1, v);
        }

        return (r1, v1);
    }

    ///  $\sqrt{x}$  modulo an irreducible `modulus` of degree t:  $x^{(2^{(mt-1)})}$ , since
the quotient ring
    /// is then the field  $\text{GF}(2^{(mt)})$ .
    pub fn sqrt_x_mod(modulus: &Poly, field: &Field) -> Poly {
        let t = modulus.degree().expect("modulus must not be zero");

```

```

        let mut root = Self::x().rem(modulus, field);
        for _ in 1..field.m() as usize * t {
            root = root.square_mod(modulus, field);
        }

        return root;
    }

    ///  $\sqrt{\text{self}}$  modulo an irreducible `modulus`, given  $\text{sqrt\_x} = \sqrt{x}$  from
    `sqrt_x_mod`.
    pub fn sqrt_mod(&self, modulus: &Poly, sqrt_x: &Poly, field: &Field) ->
    Poly {
        // self = even2 + x·odd2, where even and odd take the roots of the
        even and odd
        // coefficients, so  $\sqrt{\text{self}} = \text{even} + \sqrt{x} \cdot \text{odd}$ .
        let roots: Vec<Gf> = self.coeffs.iter().map(|&c|
        field.sqrt(c)).collect();
        let even =
        Self::from_coefs(roots.iter().step_by(2).copied().collect());
        let odd =
        Self::from_coefs(roots.iter().skip(1).step_by(2).copied().collect());

        return even.add(&sqrt_x.mul(&odd, field)).rem(modulus, field);
    }

    /// Ben-Or's test: a polynomial of degree n over GF(q) is irreducible iff
    it shares no
    /// factor with  $x^{(q^i)} - x$  for  $i = 1..=n/2$ , as those products hold every
    irreducible
    /// polynomial whose degree divides i.
    pub fn is_irreducible(&self, field: &Field) -> bool {
        let n = match self.degree() {
            None | Some(0) => return false,
            Some(n) => n,
        };
        let x = Self::x().rem(self, field);
        let mut h = x.clone();
        for _ in 0..n / 2 {
            //  $h \leftarrow h^q$ , with  $q = 2^m$ .
            for _ in 0..field.m() {
                h = h.square_mod(self, field);
            }
            if self.gcd(&h.add(&x), field).degree() != Some(0) {
                return false;
            }
        }

        return true;
    }

    pub fn random_monoid<R: Rng>(degree: usize, rng: &mut R, field: &Field) ->
    Poly {
        let mut coefs: Vec<Gf> = (0..degree).map(|_|
        field.random(rng)).collect();
        coefs.push(1);

        return Self::from_coefs(coefs);
    }

    /// About one monic polynomial in `degree` is irreducible, so this takes
    ~degree attempts.
    pub fn random_irreducible<R: Rng>(degree: usize, rng: &mut R, field:

```

```

&Field) -> Poly {
    assert!(
        degree >= 1,
        "irreducible polynomials have degree at least 1"
    );
    loop {
        let candidate = Self::random_monik(degree, rng, field);
        if candidate.is_irreducible(field) {
            return candidate;
        }
    }
}

/// For instance "x^2 +  $\alpha$ ·x +  $\alpha$ ^7".
pub fn display(&self, field: &Field) -> String {
    if self.is_zero() {
        return "0".to_string();
    }

    let terms: Vec<String> = self
        .coeffs
        .iter()
        .enumerate()
        .rev()
        .filter(|&(_, &c)| c != 0)
        .map(|(i, &c)| {
            let power = match i {
                0 => String::new(),
                1 => "x".to_string(),
                _ => format!("x^{i}"),
            };
            return match (c, i) {
                (_, 0) => field.display(c),
                (1, _) => power,
                _ => format!("{}", power, field.display(c)),
            };
        })
        .collect();

    return terms.join(" + ");
}

}

#[cfg(test)]
mod tests {
    use super::*;
    use rand::{SeedableRng, rngs::StdRng};

    fn rng() -> StdRng {
        return StdRng::seed_from_u64(4);
    }

    fn random_poly<R: Rng>(max_degree: usize, rng: &mut R, field: &Field) -> Poly {
        let len = rng.gen_range(0..=max_degree + 1);
        return Poly::from_coeffs((0..len).map(|_| field.random(rng)).collect());
    }

    fn random_nonzero_poly<R: Rng>(max_degree: usize, rng: &mut R, field: &Field) -> Poly {
        loop {
            let poly = random_poly(max_degree, rng, field);

```



```

        if !poly.is_zero() {
            return poly;
        }
    }
}

/// Every monic polynomial of the given degree over the field.
fn all_monic(degree: usize, field: &Field) -> Vec<Poly> {
    let q = field.size();
    return (0..q.pow(degree as u32))
        .map(|mut index| {
            let mut coeffs = Vec::with_capacity(degree + 1);
            for _ in 0..degree {
                coeffs.push((index % q) as Gf);
                index /= q;
            }
            coeffs.push(1);
            return Poly::from_coeffs(coeffs);
        })
        .collect();
}

#[test]
fn normalization_and_degree() {
    assert_eq!(Poly::zero().degree(), None);
    assert_eq!(Poly::from_coeffs(vec![0, 0]), Poly::zero());
    assert_eq!(Poly::from_coeffs(vec![5, 0, 0]).degree(), Some(0));
    assert_eq!(Poly::monomial(3, 4).degree(), Some(4));
    assert_eq!(Poly::constant(0), Poly::zero());

    let a = Poly::from_coeffs(vec![1, 2, 3]);
    assert_eq!(a.add(&a), Poly::zero());
}

#[test]
fn div_rem_reconstructs() {
    let field = Field::new(4);
    let mut rng = rng();
    for _ in 0..500 {
        let a = random_poly(12, &mut rng, &field);
        let b = random_nonzero_poly(6, &mut rng, &field);
        let (q, r) = a.div_rem(&b, &field);
        assert_eq!(q.mul(&b, &field).add(&r), a);
        assert!(r.degree() < b.degree());
    }
}

#[test]
fn mul_and_square_agree_with_eval() {
    let field = Field::new(4);
    let mut rng = rng();
    for _ in 0..100 {
        let a = random_poly(8, &mut rng, &field);
        let b = random_poly(8, &mut rng, &field);
        let product = a.mul(&b, &field);
        assert_eq!(a.square(&field), a.mul(&a, &field));
        for x in 0..16 {
            assert_eq!(
                product.eval(x, &field),
                field.mul(a.eval(x, &field), b.eval(x, &field))
            );
        }
    }
}

```

```

}

#[test]
fn gcd_finds_common_factor() {
    let field = Field::new(4);
    let mut rng = rng();
    for _ in 0..100 {
        let a = random_nonzero_poly(5, &mut rng, &field);
        let b = random_nonzero_poly(5, &mut rng, &field);
        let c = random_nonzero_poly(4, &mut rng, &field);
        let gcd = a.mul(&c, &field).gcd(&b.mul(&c, &field), &field);
        assert!(gcd.is_monic());
        assert!(gcd.rem(&c, &field).is_zero());
        assert!(a.mul(&c, &field).rem(&gcd, &field).is_zero());
    }
}

#[test]
fn counts_irreducible_polynomials() {
    // The number of monic irreducible polynomials of degree n over GF(q)
    is //  $(1/n) \cdot \sum_{d \mid n} \mu(d) \cdot q^{(n/d)}$ .
    let gf4 = Field::new(2);
    let gf16 = Field::new(4);
    let count = |degree, field: &Field| {
        all_monic(degree, field)
            .iter()
            .filter(|p| p.is_irreducible(field))
            .count()
    };
    assert_eq!(count(1, &gf4), 4);
    assert_eq!(count(2, &gf4), (16 - 4) / 2);
    assert_eq!(count(3, &gf4), (64 - 4) / 3);
    assert_eq!(count(4, &gf4), (256 - 16) / 4);
    assert_eq!(count(2, &gf16), (256 - 16) / 2);
}

#[test]
fn irreducible_without_roots() {
    let field = Field::new(4);
    let has_root = |p: &Poly| (0..16).any(|x| p.eval(x, &field) == 0);

    // Degrees 2 and 3 are irreducible exactly when they have no roots.
    for degree in [2, 3] {
        for p in all_monic(degree, &field).iter().step_by(7) {
            assert_eq!(p.is_irreducible(&field), !has_root(p));
        }
    }

    //  $x^2 + x + \alpha^7$  is irreducible over GF(16),  $x^2 + x + \alpha$  is not.
    let alpha7 = field.alpha_pow(7);
    assert!(Poly::from_coeffs(vec![alpha7, 1,
1])).is_irreducible(&field));
    assert!(Poly::from_coeffs(vec![field.alpha_pow(1), 1,
1])).is_irreducible(&field));

    // A product of two irreducible quadratics has no roots but is
    reducible.
    let mut rng = rng();
    let a = Poly::random_irreducible(2, &mut rng, &field);
    let b = Poly::random_irreducible(2, &mut rng, &field);
    let product = a.mul(&b, &field);
    assert!(!has_root(&product));
}

```

```

        assert!(!product.is_irreducible(&field));
    }

#[test]
fn random_irreducible_is_monic_irreducible() {
    let field = Field::new(8);
    let mut rng = rng();
    for degree in [1, 2, 5, 16] {
        let g = Poly::random_irreducible(degree, &mut rng, &field);
        assert_eq!(g.degree(), Some(degree));
        assert!(g.is_monic());
        assert!(g.is_irreducible(&field));
        if degree > 1 {
            assert!((0..256).all(|x| g.eval(x, &field) != 0));
        }
    }
}

#[test]
fn inverse_modulo_irreducible() {
    let field = Field::new(4);
    let mut rng = rng();
    let g = Poly::random_irreducible(5, &mut rng, &field);
    for _ in 0..200 {
        let a = random_nonzero_poly(4, &mut rng, &field);
        let inv = a.inv_mod(&g, &field).unwrap();
        assert!(inv.degree() < g.degree());
        assert_eq!(a.mul_mod(&inv, &g, &field), Poly::one());
    }
    assert_eq!(Poly::zero().inv_mod(&g, &field), None);
}

#[test]
fn inverse_needs_coprime() {
    let field = Field::new(4);
    // (x + 1)(x + α)
    let x_plus_1 = Poly::from_coeffs(vec![1, 1]);
    let x_plus_alpha = Poly::from_coeffs(vec![field.alpha_pow(1), 1]);
    let modulus = x_plus_1.mul(&x_plus_alpha, &field);
    assert_eq!(x_plus_1.inv_mod(&modulus, &field), None);
    assert!(Poly::x().inv_mod(&modulus, &field).is_some());
}

#[test]
fn sqrt_modulo_irreducible() {
    let field = Field::new(4);
    let mut rng = rng();
    for t in [2, 3, 6] {
        let g = Poly::random_irreducible(t, &mut rng, &field);
        let sqrt_x = Poly::sqrt_x_mod(&g, &field);
        assert_eq!(sqrt_x.square_mod(&g, &field), Poly::x().rem(&g,
&field));
        for _ in 0..100 {
            let a = random_poly(t - 1, &mut rng, &field);
            let root = a.sqrt_mod(&g, &sqrt_x, &field);
            assert_eq!(root.square_mod(&g, &field), a);
        }
    }
}

#[test]
fn partial_ext_gcd_bounds() {
    let field = Field::new(4);

```

```

    let mut rng = rng();
    for t in [2, 3, 8, 9] {
        let g = Poly::random_irreducible(t, &mut rng, &field);
        for _ in 0..100 {
            let b = random_nonzero_poly(t - 1, &mut rng, &field);
            let (r, v) = b.partial_ext_gcd(&g, t / 2, &field);
            assert_eq!(r, v.mul_mod(&b, &g, &field));
            assert!(r.degree().is_none_or(|d| d <= t / 2));
            assert!(v.degree().unwrap() <= (t - 1) / 2);
        }
    }
}

#[test]
fn display() {
    let field = Field::new(4);
    let g = Poly::from_coeffs(vec![field.alpha_pow(7), 1, 1]);
    assert_eq!(g.display(&field), "x^2 + x + α^7");
    let h = Poly::from_coeffs(vec![0, field.alpha_pow(1), 0,
field.alpha_pow(3)]);
    assert_eq!(h.display(&field), "α^3·x^3 + α·x");
    assert_eq!(Poly::zero().display(&field), "0");
    assert_eq!(Poly::one().display(&field), "1");
}
}

```

Листинг А.3 – Векторы, матрицы и перестановки над GF(2)

```

use rand::Rng;
use rand::seq::SliceRandom;
use std::ops::Range;

const WORD: usize = 64;

/// A vector over GF(2). Bit i lives in words[i / 64] at position i % 64, and
the unused bits
/// of the last word are always zero, so equality and weight can work word by
word.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct BitVec {
    len: usize,
    words: Vec<u64>,
}

impl BitVec {
    pub fn zeros(len: usize) -> Self {
        return Self {
            len,
            words: vec![0; len.div_ceil(WORD)],
        };
    }

    pub fn random<R: Rng>(len: usize, rng: &mut R) -> Self {
        let mut v = Self::zeros(len);
        for word in &mut v.words {
            *word = rng.r#gen();
        }
        v.clear_tail();

        return v;
    }
}

/// A random vector with exactly `weight` ones.

```

```

    pub fn random_with_weight<R: Rng>(len: usize, weight: usize, rng: &mut R)
-> Self {
    assert!(weight <= len, "weight {weight} exceeds length {len}");
    let mut v = Self::zeros(len);
    for i in rand::seq::index::sample(rng, len, weight) {
        v.set(i, true);
    }

    return v;
}

/// Parses a string of '0' and '1', ignoring whitespace.
pub fn from_bit_string(bits: &str) -> Result<Self, String> {
    let bits: Vec<char> = bits.chars().filter(|c| !
c.is_whitespace()).collect();
    let mut v = Self::zeros(bits.len());
    for (i, c) in bits.into_iter().enumerate() {
        match c {
            '0' => {},
            '1' => v.set(i, true),
            _ => return Err(format!("{c:?} is not a bit")),
        }
    }

    return Ok(v);
}

pub fn to_bit_string(&self) -> String {
    return (0..self.len)
        .map(|i| if self.get(i) { '1' } else { '0' })
        .collect();
}

fn clear_tail(&mut self) {
    let used = self.len % WORD;
    if used != 0 {
        *self.words.last_mut().unwrap() &= (1 << used) - 1;
    }
}

pub fn len(&self) -> usize {
    return self.len;
}

pub fn is_empty(&self) -> bool {
    return self.len == 0;
}

pub fn get(&self, i: usize) -> bool {
    assert!(i < self.len, "bit {i} out of range for length {}",
self.len);

    return self.words[i / WORD] >> (i % WORD) & 1 == 1;
}

pub fn set(&mut self, i: usize, value: bool) {
    assert!(i < self.len, "bit {i} out of range for length {}",
self.len);
    let mask = 1 << (i % WORD);
    if value {
        self.words[i / WORD] |= mask;
    } else {
        self.words[i / WORD] &= !mask;
    }
}

```

```

    }
}

pub fn flip(&mut self, i: usize) {
    assert!(i < self.len, "bit {i} out of range for length {}",
self.len);
    self.words[i / WORD] ^= 1 << (i % WORD);
}

/// self += other over GF(2).
pub fn xor_assign(&mut self, other: &BitVec) {
    assert_eq!(self.len, other.len, "length mismatch");
    for (a, b) in self.words.iter_mut().zip(&other.words) {
        *a ^= b;
    }
}

pub fn xor(&self, other: &BitVec) -> BitVec {
    let mut result = self.clone();
    result.xor_assign(other);

    return result;
}

/// Hamming weight: the number of ones.
pub fn weight(&self) -> usize {
    return self.words.iter().map(|w| w.count_ones() as usize).sum();
}

pub fn is_zero(&self) -> bool {
    return self.words.iter().all(|&w| w == 0);
}

/// Positions of the ones, in increasing order.
pub fn iter_ones(&self) -> impl Iterator<Item = usize> + ' _ {
    return self.words.iter().enumerate().flat_map(|(i, &word)| {
        let mut rest = word;
        return std::iter::from_fn(move || {
            if rest == 0 {
                return None;
            }
            let bit = rest.trailing_zeros() as usize;
            rest &= rest - 1;
            return Some(i * WORD + bit);
        });
    });
}

/// The first `len` bits.
pub fn prefix(&self, len: usize) -> BitVec {
    assert!(len <= self.len, "prefix longer than the vector");
    let mut v = Self {
        len,
        words: self.words[..len.div_ceil(WORD)].to_vec(),
    };
    v.clear_tail();

    return v;
}

pub fn slice(&self, range: Range<usize>) -> BitVec {
    assert!(range.end <= self.len, "slice out of bounds");
    let mut v = Self::zeros(range.len());

```

```

        for i in range.clone().filter(|&i| self.get(i)) {
            v.set(i - range.start, true);
        }

        return v;
    }

    /// Position of the last one, if any.
    pub fn last_one(&self) -> Option<usize> {
        let (i, word) = self
            .words
            .iter()
            .enumerate()
            .rev()
            .find(|(_, w)| **w != 0)?;

        return Some(i * WORD + (WORD - 1 - word.leading_zeros() as usize));
    }

    /// `len` bits from [len/8] bytes, least significant bit first: bit 8j +
    b is bit b of
    /// bytes[j]. The bits of the last byte past `len` must be zero.
    pub fn from_bytes(bytes: &[u8], len: usize) -> Result<Self, String> {
        if bytes.len() != len.div_ceil(8) {
            return Err(format!(
                "{len} bits need {} bytes, got {}",
                len.div_ceil(8),
                bytes.len()
            ));
        }

        let mut v = Self::zeros(len);
        for (j, &byte) in bytes.iter().enumerate() {
            v.words[j / 8] |= (byte as u64) << (8 * (j % 8));
        }
        let weight = v.weight();
        v.clear_tail();
        if v.weight() != weight {
            return Err("bits set past the end of the vector".to_string());
        }

        return Ok(v);
    }

    /// The inverse of `from_bytes`; the unused bits of the last byte are
    zero.
    pub fn to_bytes(&self) -> Vec<u8> {
        return (0..self.len.div_ceil(8))
            .map(|j| (self.words[j / 8] >> (8 * (j % 8))) as u8)
            .collect();
    }
}

/// A permutation of n positions: position i moves to map[i].
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Permutation {
    map: Vec<usize>,
}

impl Permutation {
    pub fn identity(n: usize) -> Self {
        return Self {
            map: (0..n).collect(),
        }
    }
}

```

```

    };
}

/// Fisher-Yates shuffle.
pub fn random<R: Rng>(n: usize, rng: &mut R) -> Self {
    let mut map: Vec<usize> = (0..n).collect();
    map.shuffle(rng);

    return Self { map };
}

pub fn from_vec(map: Vec<usize>) -> Result<Self, String> {
    let mut seen = vec![false; map.len()];
    for &target in &map {
        if target >= map.len() || seen[target] {
            return Err(format!("{map:?} is not a permutation"));
        }
        seen[target] = true;
    }

    return Ok(Self { map });
}

pub fn len(&self) -> usize {
    return self.map.len();
}

pub fn is_empty(&self) -> bool {
    return self.map.is_empty();
}

pub fn map(&self, i: usize) -> usize {
    return self.map[i];
}

pub fn as_slice(&self) -> &[usize] {
    return &self.map;
}

pub fn inverse(&self) -> Permutation {
    let mut map = vec![0; self.map.len()];
    for (i, &target) in self.map.iter().enumerate() {
        map[target] = i;
    }

    return Self { map };
}

/// v·P: bit i of v ends up at position map[i].
pub fn apply(&self, v: &BitVec) -> BitVec {
    assert_eq!(v.len(), self.len(), "length mismatch");
    let mut result = BitVec::zeros(v.len());
    for i in v.iter_ones() {
        result.set(self.map[i], true);
    }

    return result;
}

/// The same rearrangement as `apply`, for any sequence.
pub fn permute<T: Clone>(&self, items: &[T]) -> Vec<T> {
    assert_eq!(items.len(), self.len(), "length mismatch");
    let mut result = items.to_vec();

```



```

        for (i, item) in items.iter().enumerate() {
            result[self.map[i]] = item.clone();
        }

        return result;
    }
}

/// A matrix over GF(2), stored as a list of rows.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Matrix {
    cols: usize,
    rows: Vec<BitVec>,
}

/// The result of `Matrix::systematic`.
pub struct Systematic {
    /// rank × n, of the form [A | I_rank].
    pub matrix: Matrix,
    /// Where each column of the original matrix went.
    pub permutation: Permutation,
}

impl Matrix {
    pub fn zeros(rows: usize, cols: usize) -> Self {
        return Self {
            cols,
            rows: vec![BitVec::zeros(cols); rows],
        };
    }

    pub fn identity(n: usize) -> Self {
        let mut matrix = Self::zeros(n, n);
        for i in 0..n {
            matrix.rows[i].set(i, true);
        }

        return matrix;
    }

    pub fn random<R: Rng>(rows: usize, cols: usize, rng: &mut R) -> Self {
        return Self {
            cols,
            rows: (0..rows).map(|_| BitVec::random(cols, rng)).collect(),
        };
    }

    pub fn from_rows(cols: usize, rows: Vec<BitVec>) -> Self {
        assert!(rows.iter().all(|r| r.len() == cols), "row length mismatch");

        return Self { cols, rows };
    }

    pub fn row_count(&self) -> usize {
        return self.rows.len();
    }

    pub fn col_count(&self) -> usize {
        return self.cols;
    }

    pub fn row(&self, i: usize) -> &BitVec {
        return &self.rows[i];
    }
}

```

```

    }

    pub fn rows(&self) -> &[BitVec] {
        return &self.rows;
    }

    pub fn get(&self, row: usize, col: usize) -> bool {
        return self.rows[row].get(col);
    }

    pub fn set(&mut self, row: usize, col: usize, value: bool) {
        self.rows[row].set(col, value);
    }

    /// v·M: the sum of the rows picked out by the ones of v.
    pub fn vec_mul(&self, v: &BitVec) -> BitVec {
        assert_eq!(v.len(), self.row_count(), "length mismatch");
        let mut result = BitVec::zeros(self.cols);
        for i in v.iter_ones() {
            result.xor_assign(&self.rows[i]);
        }

        return result;
    }

    /// self · other.
    pub fn mul(&self, other: &Matrix) -> Matrix {
        return Self {
            cols: other.cols,
            rows: self.rows.iter().map(|row| other.vec_mul(row)).collect(),
        };
    }

    pub fn transpose(&self) -> Matrix {
        let mut result = Self::zeros(self.cols, self.row_count());
        for (i, row) in self.rows.iter().enumerate() {
            for j in row.iter_ones() {
                result.rows[j].set(i, true);
            }
        }

        return result;
    }

    /// M·P: column i moves to column p.map(i).
    pub fn permute_columns(&self, p: &Permutation) -> Matrix {
        return Self {
            cols: self.cols,
            rows: self.rows.iter().map(|row| p.apply(row)).collect(),
        };
    }

    pub fn columns(&self, range: Range<usize>) -> Matrix {
        assert!(range.end <= self.cols, "column range out of bounds");
        let mut result = Self::zeros(self.row_count(), range.len());
        for (i, row) in self.rows.iter().enumerate() {
            for j in row.iter_ones().filter(|j| range.contains(j)) {
                result.rows[i].set(j - range.start, true);
            }
        }

        return result;
    }
}

```

```

    /// [self | other].
    pub fn hstack(&self, other: &Matrix) -> Matrix {
        assert_eq!(self.row_count(), other.row_count(), "row count
mismatch");
        let mut result = Self::zeros(self.row_count(), self.cols +
other.cols);
        for (i, (left, right)) in
self.rows.iter().zip(&other.rows).enumerate() {
            for j in left.iter_ones() {
                result.rows[i].set(j, true);
            }
            for j in right.iter_ones() {
                result.rows[i].set(self.cols + j, true);
            }
        }

        return result;
    }

    /// Gauss-Jordan elimination on [M | I]; None when M is singular.
    pub fn inverse(&self) -> Option<Matrix> {
        let n = self.row_count();
        assert_eq!(n, self.cols, "only square matrices have inverses");

        let mut left = self.rows.clone();
        let mut right = Self::identity(n).rows;
        for col in 0..n {
            let found = (col..n).find(|&r| left[r].get(col))?;
            left.swap(col, found);
            right.swap(col, found);

            let (pivot_left, pivot_right) = (left[col].clone(),
right[col].clone());
            for r in 0..n {
                if r != col && left[r].get(col) {
                    left[r].xor_assign(&pivot_left);
                    right[r].xor_assign(&pivot_right);
                }
            }
        }

        return Some(Self {
            cols: n,
            rows: right,
        });
    }

    /// A random invertible  $n \times n$  matrix together with its inverse. About 29%
of random
    /// matrices over GF(2) are invertible, so this takes 3-4 attempts on
average.
    pub fn random_invertible<R: Rng>(n: usize, rng: &mut R) -> (Matrix,
Matrix) {
        loop {
            let candidate = Self::random(n, n, rng);
            if let Some(inverse) = candidate.inverse() {
                return (candidate, inverse);
            }
        }
    }

    /// Row-reduces to [A | I_r] up to a column permutation, where r is the

```

```

rank; zero rows
    /// are dropped. Pivots are sought in the rightmost columns first, left
    to right, so a
    /// matrix whose right  $r \times r$  block is already invertible keeps its column
    order.
    pub fn systematic(&self) -> Systematic {
        let n = self.cols;
        let mut rows = self.rows.clone();
        let right = n.saturating_sub(rows.len());

        // pivots[j] is the column holding the leading one of row j.
        let mut pivots: Vec<usize> = vec![];
        for col in (right..n).chain((0..right).rev()) {
            let rank = pivots.len();
            if rank == rows.len() {
                break;
            }
            let Some(found) = (rank..rows.len()).find(|&r| rows[r].get(col))
        else {
            continue;
        };
        rows.swap(rank, found);

        let pivot = rows[rank].clone();
        for (r, row) in rows.iter_mut().enumerate() {
            if r != rank && row.get(col) {
                row.xor_assign(&pivot);
            }
        }
        pivots.push(col);
    }

    // Every row past the rank has been cleared by now.
    let rank = pivots.len();
    rows.truncate(rank);

    // Free columns go first in their original order, then row j's pivot
    column to k + j.
    let k = n - rank;
    let mut map = vec![usize::MAX; n];
    for (j, &col) in pivots.iter().enumerate() {
        map[col] = k + j;
    }
    for (target, next) in map.iter_mut().filter(|t| **t ==
    usize::MAX).zip(0..) {
        *target = next;
    }
    let permutation = Permutation { map };

    return Systematic {
        matrix: Self {
            cols: n,
            rows: rows.iter().map(|row|
        permutation.apply(row)).collect(),
        },
        permutation,
    };
}

#[cfg(test)]
mod tests {
    use super::*;

```

```

use rand::{SeedableRng, rngs::StdRng};

fn rng() -> StdRng {
    return StdRng::seed_from_u64(4);
}

fn matrix(rows: &[&str]) -> Matrix {
    let rows: Vec<BitVec> = rows
        .iter()
        .map(|r| BitVec::from_bit_string(r).unwrap())
        .collect();

    return Matrix::from_rows(rows[0].len(), rows);
}

/// Checks everything the Goppa code relies on: the reduced matrix is  $[A \mid I]$ , and
///  $G = [I \mid A^T]$ , moved back to the original column order, is orthogonal
to the input.
fn check_systematic(h: &Matrix) -> Systematic {
    let sys = h.systematic();
    let n = h.col_count();
    let rank = sys.matrix.row_count();
    let k = n - rank;

    assert_eq!(sys.matrix.columns(k..n), Matrix::identity(rank));

    let g =
Matrix::identity(k).hstack(&sys.matrix.columns(0..k).transpose());
    assert_eq!(g.mul(&sys.matrix.transpose()), Matrix::zeros(k, rank));

    let g_original = g.permute_columns(&sys.permutation.inverse());
    assert_eq!(
        g_original.mul(&h.transpose()),
        Matrix::zeros(k, h.row_count())
    );

    return sys;
}

#[test]
fn bit_vec_basics() {
    let mut v = BitVec::zeros(70);
    assert!(v.is_zero());
    v.set(0, true);
    v.set(64, true);
    v.flip(69);
    v.flip(64);
    assert!(v.get(0) && !v.get(64) && v.get(69));
    assert_eq!(v.weight(), 2);
    assert_eq!(v.iter_ones().collect::<Vec<_>>(), vec![0, 69]);

    let s = "1011010";
    assert_eq!(BitVec::from_bit_string(s).unwrap().to_bit_string(), s);
    assert_eq!(BitVec::from_bit_string("10 11").unwrap().len(), 4);
    assert!(BitVec::from_bit_string("102").is_err());
}

#[test]
fn bit_vec_tail_stays_clear() {
    let mut rng = rng();
    for len in [1, 63, 64, 65, 130] {
        let v = BitVec::random(len, &mut rng);
    }
}

```

```

        assert!(v.weight() <= len);
        for k in [0, len / 2, len] {
            let prefix = v.prefix(k);
            assert_eq!(prefix.len(), k);
            assert!(prefix.iter_ones().all(|i| v.get(i)));
            assert_eq!(prefix.weight(), v.iter_ones().filter(|&i| i <
k).count());
        }
    }

    }

#[test]
fn bytes_round_trip() {
    let v = BitVec::from_bytes(&[0b0000_0101, 0b1000_0000], 16).unwrap();
    assert_eq!(v.iter_ones().collect::<Vec<_>>(), vec![0, 2, 15]);
    assert_eq!(v.to_bytes(), vec![0b0000_0101, 0b1000_0000]);

    let mut rng = rng();
    for len in [0, 1, 7, 8, 9, 63, 64, 65, 200] {
        let v = BitVec::random(len, &mut rng);
        let bytes = v.to_bytes();
        assert_eq!(bytes.len(), len.div_ceil(8));
        assert_eq!(BitVec::from_bytes(&bytes, len).unwrap(), v);
    }

    assert!(BitVec::from_bytes(&[0xFF], 7).is_err());
    assert!(BitVec::from_bytes(&[0x7F], 7).is_ok());
    assert!(BitVec::from_bytes(&[0, 0], 8).is_err());
}

#[test]
fn slice_and_last_one() {
    let v = BitVec::from_bit_string("0011010010").unwrap();
    assert_eq!(v.slice(2..7).to_bit_string(), "11010");
    assert_eq!(v.slice(3..3).len(), 0);
    assert_eq!(v.last_one(), Some(8));
    assert_eq!(BitVec::zeros(100).last_one(), None);

    let mut rng = rng();
    let v = BitVec::random(300, &mut rng);
    assert_eq!(v.last_one(), v.iter_ones().last());
    assert_eq!(v.slice(0..150), v.prefix(150));
    for i in 0..200 {
        assert_eq!(v.slice(50..250).get(i), v.get(50 + i));
    }
}

#[test]
fn random_with_weight_is_exact() {
    let mut rng = rng();
    for (len, weight) in [(16, 0), (16, 2), (16, 16), (1024, 50)] {
        for _ in 0..20 {
            assert_eq!(
                BitVec::random_with_weight(len, weight, &mut
rng).weight(),
                weight
            );
        }
    }
}

#[test]
fn permutation_round_trip() {

```

```

    let mut rng = rng();
    let p = Permutation::random(100, &mut rng);
    let mut sorted = p.as_slice().to_vec();
    sorted.sort();
    assert_eq!(sorted, (0..100).collect::<Vec<_>>());

    let v = BitVec::random(100, &mut rng);
    assert_eq!(p.inverse().apply(&p.apply(&v)), v);
    assert_eq!(p.apply(&v).weight(), v.weight());

    let bits: Vec<bool> = (0..100).map(|i| v.get(i)).collect();
    let moved = p.apply(&v);
    assert_eq!(
        p.permute(&bits),
        (0..100).map(|i| moved.get(i)).collect::<Vec<_>>()
    );

    assert!(Permutation::from_vec(vec![2, 0, 1]).is_ok());
    assert!(Permutation::from_vec(vec![0, 0, 2]).is_err());
    assert!(Permutation::from_vec(vec![0, 3]).is_err());
}

#[test]
fn products() {
    let mut rng = rng();
    let a = Matrix::random(7, 70, &mut rng);
    let b = Matrix::random(70, 9, &mut rng);
    let v = BitVec::random(7, &mut rng);

    assert_eq!(b.vec_mul(&a.vec_mul(&v)), a.mul(&b).vec_mul(&v));
    assert_eq!(a.mul(&b).transpose(), b.transpose().mul(&a.transpose()));
    assert_eq!(a.transpose().transpose(), a);
    assert_eq!(Matrix::identity(7).mul(&a), a);
    assert_eq!(a.mul(&Matrix::identity(70)), a);
}

#[test]
fn column_permutation_matches_vector_permutation() {
    let mut rng = rng();
    let m = Matrix::random(8, 20, &mut rng);
    let p = Permutation::random(20, &mut rng);
    let v = BitVec::random(8, &mut rng);
    assert_eq!(m.permute_columns(&p).vec_mul(&v),
p.apply(&m.vec_mul(&v)));
}

#[test]
fn columns_and_hstack() {
    let m = matrix(&["10110", "01011"]);
    assert_eq!(m.columns(0..2), matrix(&["10", "01"]));
    assert_eq!(m.columns(2..5), matrix(&["110", "011"]));
    assert_eq!(m.columns(0..2).hstack(&m.columns(2..5)), m);
}

#[test]
fn inverse() {
    let mut rng = rng();
    for n in [1, 5, 64, 100] {
        let (m, inv) = Matrix::random_invertible(n, &mut rng);
        assert_eq!(m.mul(&inv), Matrix::identity(n));
        assert_eq!(inv.mul(&m), Matrix::identity(n));
    }
}

```

```

    assert_eq!(matrix(&["110", "011", "101"]).inverse(), None);
    assert_eq!(Matrix::zeros(3, 3).inverse(), None);
}

#[test]
fn systematic_keeps_ready_matrices() {
    let h = matrix(&["1101100", "1011010", "0111001"]);
    let sys = check_systematic(&h);
    assert_eq!(sys.permutation, Permutation::identity(7));
    assert_eq!(sys.matrix, h);
}

#[test]
fn systematic_random() {
    let mut rng = rng();
    for (rows, cols) in [(8, 16), (32, 64), (40, 100)] {
        for _ in 0..20 {
            check_systematic(&Matrix::random(rows, cols, &mut rng));
        }
    }
}

#[test]
fn systematic_with_singular_right_block() {
    // The last column is zero, so a pivot has to come from the left
part.
    let h = matrix(&["1101100", "1011010", "0111000"]);
    let sys = check_systematic(&h);
    assert_eq!(sys.matrix.row_count(), 3);
    assert_ne!(sys.permutation, Permutation::identity(7));
}

#[test]
fn systematic_drops_dependent_rows() {
    let mut rng = rng();
    let mut rows = Matrix::random(6, 20, &mut rng).rows().to_vec();
    rows.push(rows[0].xor(&rows[1]));
    rows.push(BitVec::zeros(20));
    let h = Matrix::from_rows(20, rows);

    let sys = check_systematic(&h);
    assert!(sys.matrix.row_count() <= 6);
    assert_eq!(
        sys.matrix.row_count(),
        h.transpose().systematic().matrix.row_count()
    );
}
}

```

Листинг А.4 – Двоичный код Гоппы и декодер Паттерсона

```

use super::gf::{Field, Gf, MAX_M, MIN_M};
use super::matrix::{BitVec, Matrix};
use super::poly::Poly;
use rand::Rng;
use rand::seq::SliceRandom;

/// A binary Goppa code  $\Gamma(L, g)$ : the vectors  $c \in \text{GF}(2)^n$  with  $\sum c_i / (x - \alpha_i) \equiv 0 \pmod{g}$ ,
/// where  $L = (\alpha_1, \dots, \alpha_n)$  are distinct elements of  $\text{GF}(2^m)$  and  $g$  is
/// irreducible of degree  $t$ .
/// It corrects  $t$  errors with Patterson's algorithm.
pub struct Goppa {

```



```

    field: Field,
    /// Ordered so that the generator matrix is systematic,  $[I_k \mid A^T]$ .
    support: Vec<Gf>,
    g: Poly,
    ///  $\sqrt{x} \bmod g$ , for the square root in Patterson's algorithm.
    sqrt_x: Poly,
    ///  $\text{syndrome\_terms}[i] = (x - \alpha_i)^{-1} \bmod g$ , the contribution of an error in
    position  $i$ .
    syndrome_terms: Vec<Poly>,
    generator: Matrix,
}

impl Goppa {
    /// A random code:  $n$  distinct support elements and a random irreducible  $g$ 
    of degree  $t$ .
    pub fn generate<R: Rng>(m: u32, n: usize, t: usize, rng: &mut R) ->
    Result<Self, String> {
        if !(MIN_M..=MAX_M).contains(&m) {
            return Err(format!("m must be in {MIN_M}..={MAX_M}"));
        }
        let field = Field::new(m);
        if n > field.size() {
            return Err(format!("n = {n} exceeds the field size  $2^{\{m\}}$ "));
        }
        if t < 2 {
            return Err("t must be at least 2".to_string());
        }
        if m as usize * t >= n {
            return Err(format!(
                "m * t = {} leaves no room for data in n = {n}",
                m as usize * t
            ));
        }

        let mut elements: Vec<Gf> = (0..field.size() as Gf).collect();
        elements.shuffle(rng);
        elements.truncate(n);
        let g = Poly::random_irreducible(t, rng, &field);

        return Self::from_parts(field, elements, g);
    }

    /// Builds the code from a given support and Goppa polynomial. The
    support may come back
    /// reordered: see `support`.
    pub fn from_parts(field: Field, support: Vec<Gf>, g: Poly) ->
    Result<Self, String> {
        let n = support.len();
        let mut seen = vec![false; field.size()];
        for &alpha in &support {
            if alpha as usize >= field.size() {
                return Err(format!("{alpha} is not an element of  $GF(2^{\{\}})$ ",
                    field.m()));
            }
            if seen[alpha as usize] {
                return Err(format!(
                    "{} appears twice in the support",
                    field.display(alpha)
                ));
            }
            seen[alpha as usize] = true;
        }
        if g.degree().is_none_or(|t| t < 2) {

```

```

        return Err("g must have degree at least 2".to_string());
    }
    // Patterson's square roots need GF(2^m)[x]/g to be a field.
    Irreducibility also means
    // g has no roots, so every  $g(\alpha_i)$  is invertible.
    if !g.is_irreducible(&field) {
        return Err(format!("{}", g.display(&field)));
    }

    let sys = Self::parity_check_for(&field, &support, &g).systematic();
    let k = n - sys.matrix.row_count();
    if k == 0 {
        return Err("the code has no non-zero codewords".to_string());
    }

    // Reorder the support along with the columns, so the systematic G
    describes this code.
    let support = sys.permutation.permute(&support);
    let generator =
    Matrix::identity(k).hstack(&sys.matrix.columns(0..k).transpose());

    let syndrome_terms = support
        .iter()
        .map(|&alpha| {
            Poly::from_coeffs(vec![alpha, 1])
                .inv_mod(&g, &field)
                .expect("x -  $\alpha_i$  is coprime to an irreducible g without
roots")
        })
        .collect();
    let sqrt_x = Poly::sqrt_x_mod(&g, &field);

    return Ok(Self {
        field,
        support,
        g,
        sqrt_x,
        syndrome_terms,
        generator,
    });
}

/// The binary  $(m \cdot t) \times n$  parity-check matrix: rows  $\alpha_i^j / g(\alpha_i)$  for  $j =$ 
 $0..t$ , with every
/// entry of GF(2^m) spread over m rows, bit b of the entry in row  $j \cdot m +$ 
b.
fn parity_check_for(field: &Field, support: &[Gf], g: &Poly) -> Matrix {
    let m = field.m() as usize;
    let t = g.degree().unwrap();
    let mut h = Matrix::zeros(m * t, support.len());
    for (i, &alpha) in support.iter().enumerate() {
        let mut entry = field.inv(g.eval(alpha, field));
        for j in 0..t {
            for b in 0..m {
                h.set(j * m + b, i, entry >> b & 1 == 1);
            }
            entry = field.mul(entry, alpha);
        }
    }

    return h;
}

```

```

pub fn parity_check(&self) -> Matrix {
    return Self::parity_check_for(&self.field, &self.support, &self.g);
}

pub fn field(&self) -> &Field {
    return &self.field;
}

pub fn support(&self) -> &[Gf] {
    return &self.support;
}

pub fn g(&self) -> &Poly {
    return &self.g;
}

pub fn n(&self) -> usize {
    return self.support.len();
}

pub fn k(&self) -> usize {
    return self.generator.row_count();
}

pub fn t(&self) -> usize {
    return self.g.degree().unwrap();
}

///  $k \times n$ , of the form  $[I_k \mid A^T]$ .
pub fn generator(&self) -> &Matrix {
    return &self.generator;
}

/// message  $G$ ; the codeword starts with the message itself.
pub fn encode(&self, message: &BitVec) -> BitVec {
    return self.generator.vec_mul(message);
}

/// Recovers the message from a codeword of the systematic code.
pub fn message(&self, codeword: &BitVec) -> BitVec {
    return codeword.prefix(self.k());
}

///  $S(x) = \sum y_i / (x - \alpha_i) \bmod g$ , zero exactly for codewords.
pub fn syndrome(&self, y: &BitVec) -> Poly {
    let mut s = Poly::zero();
    for i in y.iter_ones() {
        s = s.add(&self.syndrome_terms[i]);
    }

    return s;
}

/// Patterson's algorithm: the codeword within distance  $t$  of  $y$ .
pub fn decode(&self, y: &BitVec) -> Result<BitVec, String> {
    if y.len() != self.n() {
        return Err(format!("expected {} bits, got {}", self.n(),
y.len()));
    }

    let s = self.syndrome(y);
    if s.is_zero() {
        return Ok(y.clone());
    }

```

```

    }

    let sigma = self.error_locator(&s);
    let errors: Vec<usize> = (0..self.n())
        .filter(|&i| sigma.eval(self.support[i], &self.field) == 0)
        .collect();
    if Some(errors.len()) != sigma.degree() {
        return Err(format!("more than {} errors", self.t()));
    }

    let mut codeword = y.clone();
    for i in errors {
        codeword.flip(i);
    }
    if !self.syndrome(&codeword).is_zero() {
        return Err(format!("more than {} errors", self.t()));
    }

    return Ok(codeword);
}

///  $\sigma(x) = \prod (x - \alpha_i)$  over the error positions, from the key equation  $S \cdot \sigma \equiv \sigma' \pmod{g}$ .
fn error_locator(&self, s: &Poly) -> Poly {
    let (field, g) = (&self.field, &self.g);

    // Split  $\sigma = a^2 + x \cdot b^2$ , so that  $\sigma' = b^2$  and the key equation becomes
    //  $a \equiv b \cdot \tau$  with  $\tau = \sqrt{(S^{-1} + x)}$ . Should  $S^{-1} = x$ , then  $\tau = 0$  and this still yields  $\sigma = x$ .
    let s_inv = s
        .inv_mod(g, field)
        .expect("a non-zero syndrome is invertible modulo an irreducible g");
    let tau = s_inv
        .add(&Poly::x())
        .rem(g, field)
        .sqrt_mod(g, &self.sqrt_x, field);

    // Stopping at  $\deg a \leq t/2$  also bounds  $\deg b \leq (t - 1)/2$ , so  $\deg \sigma \leq t$ .
    let (a, b) = tau.partial_ext_gcd(g, self.t() / 2, field);

    return a.square(field).add(&Poly::x()).mul(&b.square(field), field);
}

#[cfg(test)]
mod tests {
    use super::*;
    use rand::{SeedableRng, rngs::StdRng};

    fn rng() -> StdRng {
        return StdRng::seed_from_u64(4);
    }

    /// Encodes a random message, adds `errors` random errors and checks the round trip.
    fn check_round_trip<R: Rng>(code: &Goppa, errors: usize, rng: &mut R) {
        let message = BitVec::random(code.k(), rng);
        let codeword = code.encode(&message);
        let e = BitVec::random_with_weight(code.n(), errors, rng);
        let decoded = code.decode(&codeword.xor(&e)).unwrap();
    }
}

```

```

        assert_eq!(decoded, codeword);
        assert_eq!(code.message(&decoded), message);
    }

    #[test]
    fn code_parameters() {
        let mut rng = rng();
        for (m, n, t) in [(4, 16, 2), (5, 32, 3), (8, 256, 8), (10, 1024,
50)] {
            let code = Goppa::generate(m, n, t, &mut rng).unwrap();
            assert_eq!(code.n(), n);
            assert_eq!(code.t(), t);
            assert!(code.k() >= n - m as usize * t);

            let mut sorted = code.support().to_vec();
            sorted.sort();
            sorted.dedup();
            assert_eq!(sorted.len(), n);

            //  $G \cdot H^T = 0$ , and the rows of  $G$  have zero syndrome polynomials.
            let h = code.parity_check();
            assert_eq!(
                code.generator().mul(&h.transpose()),
                Matrix::zeros(code.k(), h.row_count())
            );
            for row in code.generator().rows() {
                assert!(code.syndrome(row).is_zero());
            }
        }
    }

    #[test]
    fn corrects_up_to_t_errors() {
        let mut rng = rng();
        for (m, n, t) in [(4, 16, 2), (4, 16, 3), (5, 32, 3), (6, 60, 5), (8,
256, 8)] {
            let code = Goppa::generate(m, n, t, &mut rng).unwrap();
            for errors in 0..=t {
                for _ in 0..50 {
                    check_round_trip(&code, errors, &mut rng);
                }
            }
        }
    }

    #[test]
    fn corrects_mceliece_1978_parameters() {
        let mut rng = rng();
        let code = Goppa::generate(10, 1024, 50, &mut rng).unwrap();
        assert_eq!(code.k(), 524);
        for errors in [0, 1, 25, 49, 50] {
            check_round_trip(&code, errors, &mut rng);
        }
    }

    #[test]
    fn minimum_distance_is_at_least_2t_plus_1() {
        let mut rng = rng();
        let code = Goppa::generate(4, 16, 2, &mut rng).unwrap();
        let k = code.k();
        assert!(k <= 12, "too many codewords to enumerate");

        let min_weight = (1..1u32 << k)

```

```

        .map(|bits| {
            let mut message = BitVec::zeros(k);
            for i in (0..k).filter(|i| bits >> i & 1 == 1) {
                message.set(i, true);
            }
            return code.encode(&message).weight();
        })
        .min()
        .unwrap();
    assert!(min_weight > 2 * code.t());
}

#[test]
fn error_at_zero_support_element() {
    // An error only where  $\alpha_i = 0$  gives  $S = 1/x$ , so  $S^{-1} = x$ : the  $\tau = 0$ 
    case.
    let mut rng = rng();
    let code = Goppa::generate(4, 16, 2, &mut rng).unwrap();
    let zero = code.support().iter().position(|&a| a == 0).unwrap();

    let codeword = code.encode(&BitVec::random(code.k(), &mut rng));
    let mut y = codeword.clone();
    y.flip(zero);
    assert_eq!(code.decode(&y).unwrap(), codeword);

    let other = (zero + 1) % code.n();
    y.flip(other);
    assert_eq!(code.decode(&y).unwrap(), codeword);
}

#[test]
fn too_many_errors_never_yield_the_original() {
    let mut rng = rng();
    let code = Goppa::generate(5, 32, 3, &mut rng).unwrap();
    let mut failures = 0;
    for _ in 0..200 {
        let codeword = code.encode(&BitVec::random(code.k(), &mut rng));
        let y = codeword.xor(&BitVec::random_with_weight(code.n(), 4,
&mut rng));
        match code.decode(&y) {
            Err(_) => failures += 1,
            Ok(other) => {
                // It can land next to another codeword, but never the
                sent one.

                assert_ne!(other, codeword);
                assert!(code.syndrome(&other).is_zero());
                assert!(other.xor(&y).weight() <= code.t());
            }
        }
    }
    assert!(failures > 0);
}

#[test]
fn from_parts_example() {
    // GF(16) with  $z^4 + z + 1$ , all 16 elements as the support,  $g = x^2 + x$ 
    +  $\alpha^7$ .
    let field = Field::new(4);
    let g = Poly::from_coeffs(vec![field.alpha_pow(7), 1, 1]);
    let code = Goppa::from_parts(field, (0..16).collect(), g).unwrap();
    assert_eq!(code.n(), 16);
    assert_eq!(code.k(), 8);
    assert_eq!(code.g().display(code.field()), "x^2 + x +  $\alpha^7$ ");
}

```

```

        let mut rng = rng();
        for errors in 0..=2 {
            check_round_trip(&code, errors, &mut rng);
        }
    }

#[test]
fn rejects_bad_parameters() {
    let mut rng = rng();
    assert!(Goppa::generate(1, 2, 2, &mut rng).is_err());
    assert!(Goppa::generate(17, 1000, 2, &mut rng).is_err());
    assert!(Goppa::generate(4, 17, 2, &mut rng).is_err());
    assert!(Goppa::generate(4, 16, 1, &mut rng).is_err());
    assert!(Goppa::generate(4, 16, 4, &mut rng).is_err());

    let field = Field::new(4);
    let irreducible = Poly::from_coeffs(vec![field.alpha_pow(7), 1, 1]);
    let reducible = Poly::from_coeffs(vec![field.alpha_pow(1), 1, 1]);
    assert!(
        Goppa::from_parts(
            Field::new(4),
            vec![0, 1, 1, 2, 3, 4, 5, 6, 7, 8],
            irreducible.clone()
        )
        .is_err()
    );
    assert!(Goppa::from_parts(Field::new(4), (0..16).collect(),
    reducible).is_err());
    assert!(Goppa::from_parts(Field::new(4), (0..16).collect(),
    Poly::x()).is_err());
    assert!(Goppa::from_parts(Field::new(4), vec![0, 1, 2],
    irreducible).is_err());

    let code = Goppa::generate(4, 16, 2, &mut rng).unwrap();
    assert!(code.decode(&BitVec::zeros(15)).is_err());
}
}

```

Листинг А.5 – Реализация криптосистемы Мак-Элиса

```

use super::goppa::Goppa;
use super::matrix::{BitVec, Matrix, Permutation};
use rand::rngs::OsRng;
use rand::{Rng, SeedableRng, rngs::StdRng};

/// Code parameters: a binary Goppa code over GF(2^m) of length n correcting
t errors.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct Params {
    pub m: u32,
    pub n: usize,
    pub t: usize,
}

impl Params {
    /// Small enough to follow by hand: k = 8.
    pub const TOY: Params = Params { m: 4, n: 16, t: 2 };
    pub const SMALL: Params = Params { m: 8, n: 256, t: 8 };
    /// The parameters of McEliece's 1978 paper: k = 524.
    pub const ORIGINAL: Params = Params {
        m: 10,
        n: 1024,
    }
}

```

```

        t: 50,
    };
}

#[derive(Clone)]
pub struct PublicKey {
    ///  $G' = S \cdot G \cdot P$ ,  $k \times n$ .
    matrix: Matrix,
    t: usize,
}

impl PublicKey {
    pub fn matrix(&self) -> &Matrix {
        return &self.matrix;
    }

    pub fn n(&self) -> usize {
        return self.matrix.col_count();
    }

    pub fn k(&self) -> usize {
        return self.matrix.row_count();
    }

    pub fn t(&self) -> usize {
        return self.t;
    }

    /// Bytes of one ciphertext block.
    fn block_bytes(&self) -> usize {
        return self.n().div_ceil(8);
    }
}

pub struct PrivateKey {
    public: PublicKey,
    code: Goppa,
    s_inv: Matrix,
    p_inv: Permutation,
}

impl PrivateKey {
    pub fn public(&self) -> &PublicKey {
        return &self.public;
    }

    pub fn code(&self) -> &Goppa {
        return &self.code;
    }

    pub fn s_inv(&self) -> &Matrix {
        return &self.s_inv;
    }

    pub fn p_inv(&self) -> &Permutation {
        return &self.p_inv;
    }
}

pub struct McEliece {}

impl McEliece {
    pub fn keygen(params: Params) -> Result<(PrivateKey, PublicKey), String>

```



```

{
    let mut rng = StdRng::from_rng(&mut OsRng).unwrap();

    return Self::keygen_with_rng(params, &mut rng);
}

pub fn keygen_with_rng<R: Rng>(
    params: Params,
    rng: &mut R,
) -> Result<(PrivateKey, PublicKey), String> {
    let code = Goppa::generate(params.m, params.n, params.t, rng)?;
    let (s, s_inv) = Matrix::random_invertible(code.k(), rng);
    let p = Permutation::random(code.n(), rng);

    let public = PublicKey {
        matrix: s.mul(code.generator()).permute_columns(&p),
        t: code.t(),
    };
    let private = PrivateKey {
        public: public.clone(),
        code,
        s_inv,
        p_inv: p.inverse(),
    };

    return Ok((private, public));
}

///  $c = m \cdot G' + e$ , with  $e$  a random vector of weight exactly  $t$ .
pub fn encrypt_block<R: Rng>(message: &BitVec, key: &PublicKey, rng: &mut
R) -> BitVec {
    let e = BitVec::random_with_weight(key.n(), key.t(), rng);

    return key.matrix.vec_mul(message).xor(&e);
}

///  $c \cdot P^{-1} = (m \cdot S) \cdot G + e \cdot P^{-1}$  still carries  $t$  errors, which the Goppa
decoder removes;
/// the codeword starts with  $m \cdot S$ , and  $S^{-1}$  recovers  $m$ .
pub fn decrypt_block(cipher: &BitVec, key: &PrivateKey) -> Result<BitVec,
String> {
    let codeword = key.code.decode(&key.p_inv.apply(cipher))?;
    let message_s = key.code.message(&codeword);

    return Ok(key.s_inv.vec_mul(&message_s));
}

pub fn encrypt(plain_bytes: Vec<u8>, key: &PublicKey) -> Vec<u8> {
    let mut rng = StdRng::from_rng(&mut OsRng).unwrap();

    return Self::encrypt_with_rng(plain_bytes, key, &mut rng);
}

/// Splits the plaintext bits into  $k$ -bit blocks after padding them with a
one and then
/// zeros; every block becomes  $\lceil n/8 \rceil$  bytes of ciphertext.
pub fn encrypt_with_rng<R: Rng>(plain_bytes: Vec<u8>, key: &PublicKey,
rng: &mut R) -> Vec<u8> {
    let k = key.k();
    let data_len = plain_bytes.len() * 8;
    let blocks = data_len / k + 1;

    let data = BitVec::from_bytes(&plain_bytes, data_len).unwrap();

```

```

        let mut padded = BitVec::zeros(blocks * k);
        for i in data.iter_ones() {
            padded.set(i, true);
        }
        padded.set(data_len, true);

        let mut result = Vec::with_capacity(blocks * key.block_bytes());
        for b in 0..blocks {
            let message = padded.slice(b * k..(b + 1) * k);
            result.extend(Self::encrypt_block(&message, key,
rng).to_bytes());
        }

        return result;
    }

    pub fn decrypt(encrypted_bytes: Vec<u8>, key: &PrivateKey) ->
Result<Vec<u8>, String> {
        let (n, k) = (key.public.n(), key.public.k());
        let block_bytes = key.public.block_bytes();
        if encrypted_bytes.is_empty() || !
encrypted_bytes.len().is_multiple_of(block_bytes) {
            return Err("Invalid message length".to_owned());
        }

        let blocks = encrypted_bytes.len() / block_bytes;
        let mut padded = BitVec::zeros(blocks * k);
        for (b, chunk) in encrypted_bytes.chunks(block_bytes).enumerate() {
            let cipher = BitVec::from_bytes(chunk, n).map_err(|_| "Block
corrupted".to_owned())?;
            let message =
                Self::decrypt_block(&cipher, key).map_err(|_| "Block
corrupted".to_owned())?;
            for i in message.iter_ones() {
                padded.set(b * k + i, true);
            }
        }

        // The padding is a one followed by zeros, 1..=k bits long, and
starts on a byte.
        match padded.last_one() {
            Some(pos) if pos.is_multiple_of(8) && pos + k >= padded.len() =>
{
                return Ok(padded.prefix(pos).to_bytes());
            }
            _ => return Err("Invalid padding".to_owned()),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn rng() -> StdRng {
        return StdRng::seed_from_u64(4);
    }

    #[test]
    fn keys_fit_together() {
        let mut rng = rng();
        let (private, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();

```

```

    let code = private.code();
    assert_eq!(
        (public.n(), public.k(), public.t()),
        (code.n(), code.k(), code.t())
    );

    //  $G' = S \cdot G \cdot P$ , with S and P taken back from their inverses.
    let s = private.s_inv().inverse().unwrap();
    let p = private.p_inv().inverse();
    assert_eq!(
        public.matrix(),
        &s.mul(code.generator()).permute_columns(&p)
    );

    //  $G'$  is not simply  $G$ : S and P hide the systematic form.
    assert_ne!(
        public.matrix().columns(0..public.k()),
        Matrix::identity(public.k())
    );
}

#[test]
fn block_round_trip() {
    let mut rng = rng();
    for params in [Params::TOY, Params::SMALL, Params::ORIGINAL] {
        let (private, public) = McEliece::keygen_with_rng(params, &mut
rng).unwrap();
        for _ in 0..20 {
            let message = BitVec::random(public.k(), &mut rng);
            let cipher = McEliece::encrypt_block(&message, &public, &mut
rng);
            assert_eq!(cipher.len(), public.n());
            assert_eq!(McEliece::decrypt_block(&cipher,
&private).unwrap(), message);
        }
    }
}

#[test]
fn error_vector_has_weight_t() {
    let mut rng = rng();
    let (_, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
    let zero = BitVec::zeros(public.k());
    for _ in 0..20 {
        // For m = 0 the ciphertext is e itself.
        assert_eq!(
            McEliece::encrypt_block(&zero, &public, &mut rng).weight(),
            public.t()
        );
    }
}

#[test]
fn bytes_round_trip() {
    let mut rng = rng();
    for params in [Params::TOY, Params::SMALL] {
        let (private, public) = McEliece::keygen_with_rng(params, &mut
rng).unwrap();
        let k = public.k();
        for len in [0, 1, 2, 3, k / 8, k / 8 + 1, 2 * k / 8, 100, 257] {
            let plain: Vec<u8> = (0..len).map(|_| rng.r#gen()).collect();
            let cipher = McEliece::encrypt_with_rng(plain.clone(),

```

```

    &public, &mut rng);
        let blocks = (len * 8) / k + 1;
        assert_eq!(cipher.len(), blocks * public.n().div_ceil(8));
        assert_eq!(McEliece::decrypt(cipher, &private).unwrap(),
plain);
    }
}

#[test]
fn original_parameters_round_trip() {
    let mut rng = rng();
    let (private, public) = McEliece::keygen_with_rng(Params::ORIGINAL,
&mut rng).unwrap();
    let plain = "Криптосистема Мак-Элиса, 1978".as_bytes().to_vec();
    let cipher = McEliece::encrypt(plain.clone(), &public);
    assert_eq!(McEliece::decrypt(cipher, &private).unwrap(), plain);
}

#[test]
fn encryption_is_randomized() {
    let mut rng = rng();
    let (private, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
    let plain = b"same text".to_vec();
    let first = McEliece::encrypt_with_rng(plain.clone(), &public, &mut
rng);
    let second = McEliece::encrypt_with_rng(plain.clone(), &public, &mut
rng);
    assert_ne!(first, second);
    assert_eq!(McEliece::decrypt(first, &private).unwrap(), plain);
    assert_eq!(McEliece::decrypt(second, &private).unwrap(), plain);
}

#[test]
fn rejects_malformed_ciphertexts() {
    let mut rng = rng();
    let (private, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
    let cipher = McEliece::encrypt_with_rng(b"hello".to_vec(), &public,
&mut rng);

    assert!(McEliece::decrypt(vec![], &private).is_err());
    assert!(McEliece::decrypt(cipher[1..].to_vec(), &private).is_err());

    // A decryptable block that carries no padding bit.
    let zero_block = McEliece::encrypt_block(&BitVec::zeros(public.k()),
&public, &mut rng);
    assert!(McEliece::decrypt(zero_block.to_bytes(), &private).is_err());

    let (other, _) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
    assert_ne!(
        McEliece::decrypt(cipher, &other).ok(),
        Some(b"hello".to_vec())
    );
}

#[test]
fn rejects_bits_past_n() {
    // n = 30 leaves two unused bits in the last byte of every block.
    let mut rng = rng();
    let params = Params { m: 5, n: 30, t: 3 };

```

```

        let (private, public) = McEliece::keygen_with_rng(params, &mut
rng).unwrap();
        let mut cipher = McEliece::encrypt_with_rng(b"x".to_vec(), &public,
&mut rng);
        assert_eq!(McEliece::decrypt(cipher.clone(), &private).unwrap(),
b"x");
        cipher[3] |= 0x80;
        assert!(McEliece::decrypt(cipher, &private).is_err());
    }

    #[test]
    fn one_flipped_bit_is_fatal_or_absorbed() {
        // Flipping a ciphertext bit either cancels one of the t errors,
which still decodes,
        // or adds a (t+1)-th, which the decoder cannot correct.
        let mut rng = rng();
        let (private, public) = McEliece::keygen_with_rng(Params::TOY, &mut
rng).unwrap();
        let message = BitVec::random(public.k(), &mut rng);
        let cipher = McEliece::encrypt_block(&message, &public, &mut rng);
        let error_positions: Vec<usize> = cipher
            .xor(&public.matrix().vec_mul(&message))
            .iter_ones()
            .collect();

        for i in 0..public.n() {
            let mut tampered = cipher.clone();
            tampered.flip(i);
            let result = McEliece::decrypt_block(&tampered, &private);
            if error_positions.contains(&i) {
                assert_eq!(result.unwrap(), message);
            } else {
                assert_ne!(result.ok(), Some(message.clone()));
            }
        }
    }
}

```

Листинг А.6 – Точка входа в программу

```

// Some helpers of the algebra modules are only exercised by their tests.
#[allow(dead_code)]
mod encryption;
use common::cli::{self, IoMode};
use encryption::gf::{MAX_M, MIN_M};
use encryption::matrix::{BitVec, Matrix};
use encryption::mceliece::*;
use rand::rngs::OsRng;
use rand::{SeedableRng, rngs::StdRng};
use std::time::Instant;

const TITLE: &str = "McEliece cryptosystem";
const ENCRYPTED_EXTENSION: &str = "mce";
const PRESETS: [(&str, Params); 3] = [
    ("toy", Params::TOY),
    ("small", Params::SMALL),
    ("original (1978)", Params::ORIGINAL),
];
/// Matrices and lists with at most this many columns or entries are printed
in full.
const PRINT_LIMIT: usize = 64;

fn generate_keys(params: Params) -> Result<(PrivateKey, PublicKey), String> {

```

```

println!(
    "Generating keys for m = {}, n = {}, t = {}...",
    params.m, params.n, params.t
);
let start = Instant::now();
let keys = McEliece::keygen(params)?;
println!("Done in {:.2?}", start.elapsed());

return Ok(keys);
}

fn parse_number(label: &str, default: usize) -> Result<usize, String> {
    let answer = cli::prompt(&format!("{label} [{default}]: "));
    if answer.is_empty() {
        return Ok(default);
    }

    return answer
        .trim()
        .parse()
        .map_err(|_| format!("{label} expects a non-negative integer"));
}

fn read_params(current: Params) -> Result<Params, String> {
    for (idx, (name, p)) in PRESETS.iter().enumerate() {
        println!(
            "{} {name:<16} m = {:<2}, n = {:<4}, t = {}",
            idx + 1,
            p.m,
            p.n,
            p.t
        );
    }
    println!("{}", PRESETS.len() + 1);

    let choice = cli::prompt("Parameters [keep current]: ");
    if choice.is_empty() {
        return Ok(current);
    }
    match choice.parse::<usize>() {
        Ok(idx) if (1..=PRESETS.len()).contains(&idx) => return
Ok(PRESETS[idx - 1].1),
        Ok(idx) if idx == PRESETS.len() + 1 => {}
        _ => return Err(format!("unknown choice: {choice:?}")),
    }

    let m = parse_number(&format!("m ({MIN_M}..={MAX_M})"), current.m as
usize)?;
    let n = parse_number(
        &format!("n (up to 2^m = {})", 1usize << m.min(MAX_M as usize)),
        current.n,
    )?;
    let t = parse_number("t", current.t)?;

    return Ok(Params { m: m as u32, n, t });
}

/// z^4 + z + 1 for 0b10011.
fn field_polynomial(modulus: u32) -> String {
    let terms: Vec<String> = (0..u32::BITS)
        .rev()
        .filter(|i| modulus >> i & 1 == 1)
        .map(|i| match i {

```

```

        0 => "1".to_string(),
        1 => "z".to_string(),
        _ => format!("z^{i}"),
    })
    .collect();

    return terms.join(" + ");
}

/// Bits in groups of eight, for readability.
fn grouped(v: &BitVec) -> String {
    let bits = v.to_bit_string();
    let groups: Vec<&str> = bits
        .as_bytes()
        .chunks(8)
        .map(|chunk| std::str::from_utf8(chunk).unwrap())
        .collect();

    return groups.join(" ");
}

fn print_matrix(label: &str, matrix: &Matrix) {
    let (rows, cols) = (matrix.row_count(), matrix.col_count());
    println!(
        "  {label:<11}: {rows} × {cols}, {} bytes",
        (rows * cols).div_ceil(8)
    );
    if cols > PRINT_LIMIT || rows > PRINT_LIMIT {
        return;
    }

    for row in matrix.rows() {
        println!("  {:<11} {}", "", grouped(row));
    }
}

fn print_list<T: ToString>(label: &str, items: &[T]) {
    let shown: Vec<String> = items
        .iter()
        .take(PRINT_LIMIT)
        .map(|item| item.to_string())
        .collect();
    let rest = items.len() - shown.len();
    let tail = if rest > 0 {
        format!("{}", ... ({} more)")
    } else {
        String::new()
    };

    println!("  {label:<11}: {}{}", shown.join(", "), tail);
}

fn print_keys(private: &PrivateKey, public: &PublicKey) {
    let code = private.code();
    let field = code.field();

    println!("Public key");
    println!(
        "  n, k, t      : {}, {}, {}",
        public.n(),
        public.k(),
        public.t()
    );
}

```

```

print_matrix("G' = S·G·P", public.matrix());

println!("Private key");
println!(
    "  field      : GF(2^{}) = GF(2)[z]/({})",
    field.m(),
    field_polynomial(field.modulus())
);
println!("  g(x)      : {}", code.g().display(field));
let support: Vec<String> = code.support().iter().map(|&a|
field.display(a)).collect();
print_list("support L", &support);
print_matrix("S-1", private.s_inv());
print_list("P-1", private.p_inv().as_slice());
}

fn run_cipher(
    io_mode: IoMode,
    encrypt: bool,
    private: &PrivateKey,
    public: &PublicKey,
) -> Result<(), String> {
    let (input, source) = cli::read_payload(io_mode, encrypt)?;
    if input.is_empty() {
        return Err("nothing to process: the input is empty".to_string());
    }

    let destination = match &source {
        Some(path) => Some(cli::ask_output_path(&cli::default_output_path(
            path,
            ENCRYPTED_EXTENSION,
            encrypt,
        ))?),
        None => None,
    };

    let output = if encrypt {
        McEliece::encrypt(input.clone(), public)
    } else {
        McEliece::decrypt(input.clone(), private)?
    };

    println!(
        "  mode      : {}",
        if encrypt { "encryption" } else { "decryption" }
    );

    return cli::emit_output(
        encrypt,
        &input,
        &output,
        source.as_deref(),
        destination.as_deref(),
    );
}

/// Encrypts and decrypts one block, printing every intermediate value.
fn trace_block(private: &PrivateKey, public: &PublicKey) -> Result<(),
String> {
    let (n, k) = (public.n(), public.k());
    let mut rng = StdRng::from_rng(&mut OsRng).unwrap();

    let answer = cli::prompt(&format!("Message ({k} bits, empty for random):

```



```

    ));
    let message = if answer.trim().is_empty() {
        BitVec::random(k, &mut rng)
    } else {
        BitVec::from_bit_string(&answer)?
    };
    if message.len() != k {
        return Err(format!("expected {k} bits, got {}", message.len()));
    }
    println!();

    let code = private.code();
    let encoded = public.matrix().vec_mul(&message);
    let cipher = McEliece::encrypt_block(&message, public, &mut rng);
    let e = cipher.xor(&encoded);
    let positions: Vec<usize> = e.iter_ones().collect();

    println!("Encryption");
    println!("  m          : {}", grouped(&message));
    println!("  m·G'       : {}", grouped(&encoded));
    println!("  e (weight {:<3}) : {}", e.weight(), grouped(&e));
    println!("  e positions   : {positions:?}");
    println!("  c = m·G' + e   : {}", grouped(&cipher));

    let unpermuted = private.p_inv().apply(&cipher);
    let syndrome = code.syndrome(&unpermuted);
    let codeword = code.decode(&unpermuted)?;
    let fixed: Vec<usize> = unpermuted.xor(&codeword).iter_ones().collect();
    let message_s = code.message(&codeword);
    let decrypted = private.s_inv().vec_mul(&message_s);

    println!("Decryption");
    println!("  c·P-1      : {}", grouped(&unpermuted));
    println!("  syndrome S(x) : {}", syndrome.display(code.field()));
    println!("  fixed at      : {fixed:?} (e·P-1)");
    println!("  m·S·G         : {}", grouped(&codeword));
    println!("  m·S (first k) : {}", grouped(&message_s));
    println!("  m = m·S·S-1 : {}", grouped(&decrypted));
    println!(
        "  result          : {}",
        if decrypted == message {
            "matches the message"
        } else {
            "MISMATCH"
        }
    );
    println!("  block sizes    : {k} bits in, {n} bits out");

    return Ok(());
}

fn main() {
    let mut params = Params::ORIGINAL;
    let (mut private, mut public) = generate_keys(params).expect("the preset parameters are valid");
    let mut io_mode = IoMode::Text;

    loop {
        println!();
        println!(
            "{TITLE} - input/output: {}, n = {}, k = {}, t = {}",
            io_mode.name(),
            public.n(),

```

```

        public.k(),
        public.t()
    );
    println!("{:<22}2) decrypt", "1) encrypt");
    println!("3) trace one block");

    let choice = cli::prompt(
        "Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to
generate keys, 'q' to quit): ",
    );
    println!();

    let result = match choice.as_str() {
        "1" => run_cipher(io_mode, true, &private, &public),
        "2" => run_cipher(io_mode, false, &private, &public),
        "3" => trace_block(&private, &public),
        "m" => {
            io_mode = io_mode.toggled();
            println!("Switched to {} mode", io_mode.name());
            Ok(())
        }
        "k" => {
            print_keys(&private, &public);
            Ok(())
        }
        "g" => read_params(params).and_then(|new_params| {
            (private, public) = generate_keys(new_params)?;
            params = new_params;
            Ok(())
        }),
        "q" => break,
        _ => Err(format!("unknown choice: {choice:?}")),
    };

    if let Err(err) = result {
        println!("Error: {err}");
    }
}
}

```

ПРИЛОЖЕНИЕ Б **(обязательное)** **Блок-схема алгоритма**

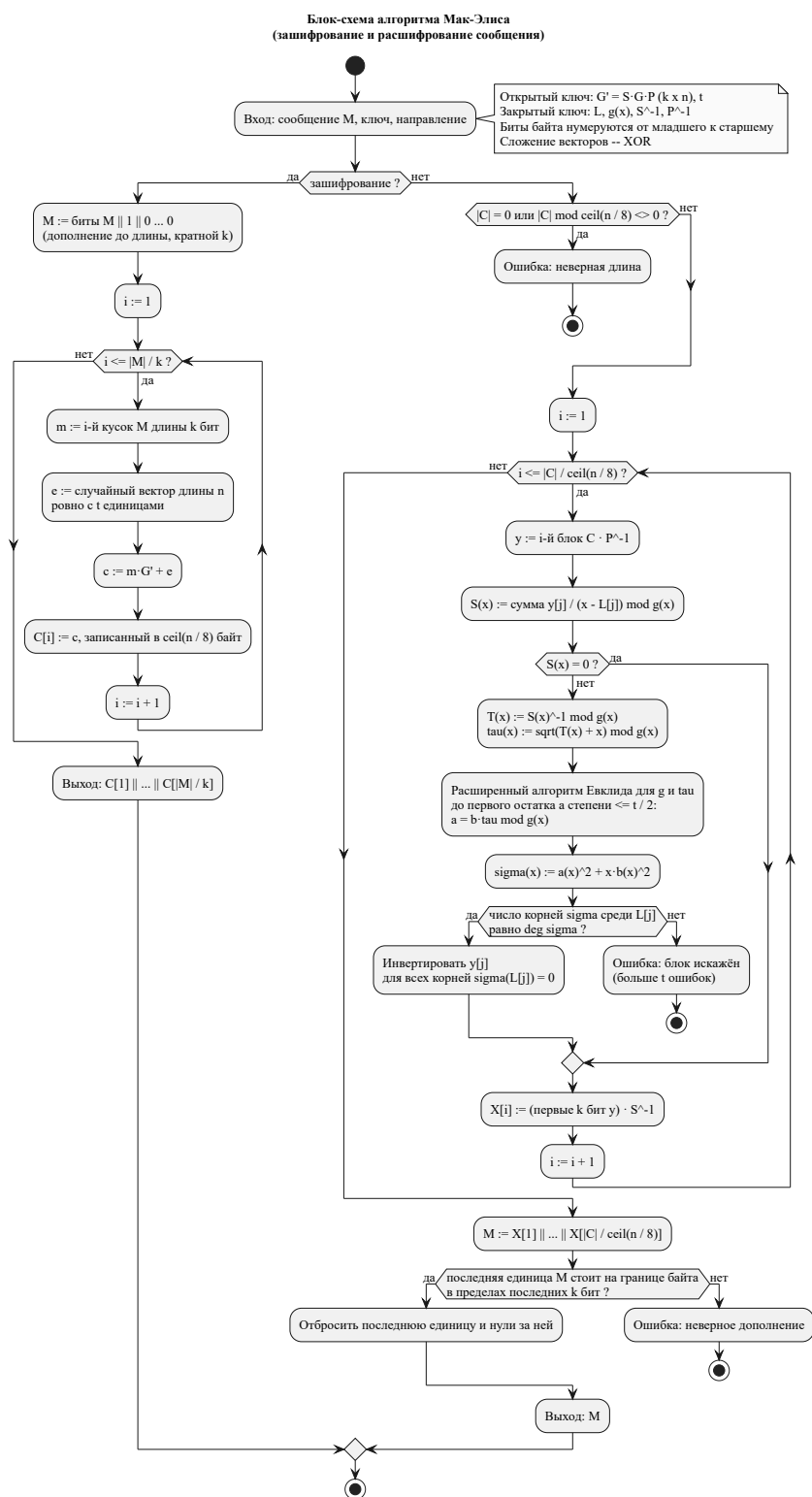


Рисунок Б.1 – Блок-схема алгоритма